

БЫСТРЫЕ АЛГОРИТМЫ

1. Умножение матриц по Винограду.

В 1987 году Дон Копперсмит и Виноград опубликовали метод, содержащий $O(n^{2.376})$ операций и улучшили его в 2011 до $O(n^{2.373})$.

Скалярное произведение, по замыслу Винограда, можно произвести:

$$c_{ij} = \sum_{k=1}^{q/2} (a_{i,2k-1} + b_{2k,j}) (a_{i,2k} + b_{2k-1,j}) - \sum_{k=1}^{q/2} a_{i,2k-1} a_{i,2k} - \sum_{k=1}^{q/2} b_{2k-1,j} b_{2k,j} \quad (3)$$

Казалось бы, это только увеличит количество арифметических операций по сравнению с классическим методом, однако Виноград предложил находить второе и третье слагаемые в (3) на предварительном этапе вычислений, заранее для каждой строки матрицы A и столбца B соответственно. Так, вычислив единожды для строки i матрицы A значение выражения $\sum_{k=1}^{q/2} a_{i,2k-1} a_{i,2k}$ его можно далее использовать m раз при нахождении элементов i -ой строки матрицы C . Ведь данное выражение участвует в определении c_{ij} по ф. (3) для выбранного i и $1 \leq j \leq m$.

Аналогично, вычислив единожды для столбца j матрицы B значение выражения $\sum_{k=1}^{q/2} b_{2k-1,j} b_{2k,j}$ его можно далее использовать n раз при нахождении элементов j -ого столбца матрицы C . Ведь данное выражение участвует в определении c_{ij} по ф. (3) для выбранного j и $1 \leq i \leq n$.

Подготовка матрицы A в методе Винограда

```
for i=1:n
    row(i)=A(i,1)A(i,2)
    for k=2:q/2
        row(i)=row(i)+A(i,2k-1)A(i,2k)
    endfor
endfor
```

Подготовка матрицы B в методе Винограда

```
for j=1:m
    col(j)=B(1,j)B(2,j)
    for k=2:q/2
        col(j)=col(j)+B(2k-1,j)B(2k,j)
    endfor
endfor
```

Умножение матриц по методу Винограда

```
for i=1:n
    for j=1:m
        C(i,j)=-row(i)-col(j)
        for k=1:q/2
            C(i,j)=C(i,j)+(A(i,2k-1)+B(2k,j)) * (A(i,2k)+B(2k-1,j))
        endfor
    endfor
endfor
```

Наглядный пример: умножение матриц 2×2 по Винограду

Исходные матрицы

$$A = \begin{pmatrix} 3 & 1 \\ 2 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 7 \\ 6 & 2 \end{pmatrix}$$

Шаг 1. Предвычисления для строк A

$$\text{rowFactor}[i] = \sum_{k=1}^1 a_{i,2k-1} \cdot a_{i,2k}$$

$$\text{rowFactor}[0] = a_{0,1} \cdot a_{0,2} = 3 \cdot 1 = 3$$

$$\text{rowFactor}[1] = a_{1,1} \cdot a_{1,2} = 2 \cdot 4 = 8$$

Шаг 2. Предвычисления для столбцов B

$$\text{colFactor}[j] = \sum_{k=1}^1 b_{2k-1,j} \cdot b_{2k,j}$$

$$\text{colFactor}[0] = b_{1,1} \cdot b_{2,1} = 5 \cdot 6 = 30$$

$$\text{colFactor}[1] = b_{1,2} \cdot b_{2,2} = 7 \cdot 2 = 14$$

Шаг 3. Вычисление c_{11}

$$c_{11} = (a_{0,1} + b_{2,1})(a_{0,2} + b_{1,1}) - \text{rowFactor}[0] - \text{colFactor}[0]$$

$$= (3 + 6)(1 + 5) - 3 - 30 = 9 \cdot 6 - 33 = 54 - 33 = 21$$

Проверка: $3 \cdot 5 + 1 \cdot 6 = 15 + 6 = 21 \square$

Шаг 4. Вычисление c_{12}

$$c_{12} = (a_{0,1} + b_{2,2})(a_{0,2} + b_{1,2}) - \text{rowFactor}[0] - \text{colFactor}[1]$$

$$= (3 + 2)(1 + 7) - 3 - 14 = 5 \cdot 8 - 17 = 40 - 17 = 23$$

Проверка: $3 \cdot 7 + 1 \cdot 2 = 21 + 2 = 23 \square$

Шаг 5. Вычисление c_{21}

$$c_{21} = (a_{1,1} + b_{2,1})(a_{1,2} + b_{1,1}) - \text{rowFactor}[1] - \text{colFactor}[0]$$

$$= (2 + 6)(4 + 5) - 8 - 30 = 8 \cdot 9 - 38 = 72 - 38 = 34$$

Проверка: $2 \cdot 5 + 4 \cdot 6 = 10 + 24 = 34 \square$

Шаг 6. Вычисление c_{22}

$$c_{22} = (a_{1,1} + b_{2,2})(a_{1,2} + b_{1,2}) - \text{rowFactor}[1] - \text{colFactor}[1]$$

$$= (2 + 2)(4 + 7) - 8 - 14 = 4 \cdot 11 - 22 = 44 - 22 = 22$$

Проверка: $2 \cdot 7 + 4 \cdot 2 = 14 + 8 = 22 \square$

Результат

$$C = \begin{pmatrix} 21 & 23 \\ 34 & 22 \end{pmatrix}$$

МОДЕЛИ В ТЕОРИИ ПАРАЛЛЕЛЬНЫХ ВЫЧИСЛЕНИЙ

2. Классификация параллельных вычислительных систем по Флинну.

Модель (классификация) Флинна 1966 г.

данные, команды → ЭВМ

Классификация:

команды/потоки	одна команда	много команд
один поток	однопроц. ЭВМ (SISD)	конвейерные ЭВМ (MISD)
много потоков	векторные (SIMD)	многопроц. ЭВМ (MIMD)



Рис. 1.4. Модель архитектуры конвейерной ЭВМ

3. Классификация параллельных вычислительных систем по Хокни.

Классификация Хокни для MIMD

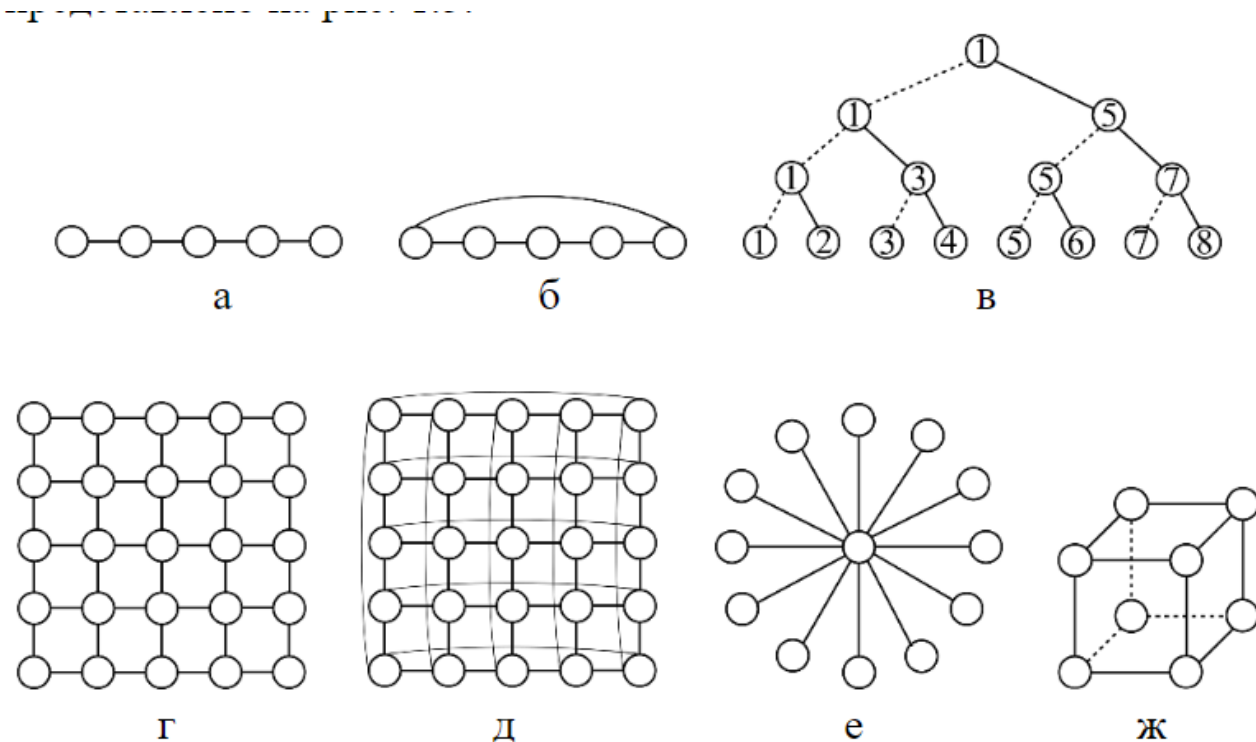


Рис. 1.5. Примеры сетевых конфигураций:

а – процессорная линия; б – процессорное кольцо; в – бинарное дерево (пунктирными линиями изображаются воображаемые связи процессоров с самими собой); г – процессорная решетка; д – тор; е – звезда; ж – гиперкуб размерности 3 (действительные связи процессоров изображаются непрерывными и пунктирными линиями)

Конвейерные ЭВМ Хокни отнес к многопроцессорным, оставив класс MISD пустым.

1. Конвейерные (!Проблема взаимного исключения!) Смысл конвейеризации состоит в одновременном исполнении разных команд над различными фрагментами одного потока данных. При этом процессоров оказывается несколько, каждый со своей памятью.
2. Переключаемые - все процессоры соединены со всеми через особое устройство – переключатель
 1. С одной памятью
 2. С распределенной памятью Переключаемые в свою очередь разделяются на ЭВМ с общей и распределенной памятью. В первом случае процессоры работают над общей памятью и коммуникации между ними осуществляются через нее. При этом в ходе вычислений необходимо регулировать очередность доступа к общим данным. Во втором – память у всех процессоров локальная и указанной проблемы не возникает, однако технически обеспечить полносвязную топологию коммуникаций при большом количестве процессоров сложно.
3. Сети - (процессор соединяется с определенным количеством соседей)
 1. Процессорная линия (частный случай конвейерного)
 2. Процессорное кольцо
 3. Процессорная решетка
 4. Тор (бублик)
 5. Гиперкуб
 6. Бинарное дерево (частный случай гиперкуба)
 7. Звезда
 8. Гиперкуб с размерностью k содержит $p = 2^k$ вершин

4. !Модели представления параллельных алгоритмов.!

По Маркову под алгоритмом будем понимать ясные предписания, задающие вычислительный процесс, идущий от варьируемых начальных данных к искомому результату

Свойства алгоритмов: - **Детерминированность (результативность)** - за конечное количество шагов алгоритм либо приводит к результату, либо к убежденности невозможности получить результат. - **Дискретность** - подразумевает, что алгоритм должен разбиваться на шаги - **Масса** - схожие задачи должны допускать решение одним алгоритмом - **Однозначность** - алгоритм не допускает двух разных трактовок

Модели параллельных алгоритмов: ##### 1. Задача/Канал

Под задачей будем понимать последовательный набор инструкций и данных, над которыми инструкции выполняются (Кружочки)

Параллельность обеспечивается за счет того, что одновременно выполняются несколько задач

Канал - средство коммуникации между задачами (Стрелочки)

Два правила каналов:

1. Перемещение данных по каналу всегда в одну сторону
2. Данные по каналу перемещаются в соответствии с правилом - первый вошел, первый вышел (Вагончики)

В ходе работы алгоритма задачи могут появляться и упраздняться

Свойства коммуникации:

1. Синхронная/Асинхронная коммуникация

Операции приема и отправки

Под **синхронной коммуникацией** будем понимать ситуацию, когда следующая за этой операцией инструкция не выполняется до окончания коммуникации

В **асинхронном** случае следующая инструкция после коммуникации выполняется сразу после начала этой коммуникации

2. Локальные/Глобальные коммуникации

p - число задач

соседи $\leq \lceil \log_2 p \rceil$ до ближайшего большего числа, то локальная

3. Статические/Динамические коммуникации

Если топология коммуникации алгоритма меняется по ходу его выполнения, то такие коммуникации называются динамические

4. Регулярные/Нерегулярные коммуникации

Если коммуникация алгоритма не отображается на стандартную сетевую топологию, то она нерегулярная

2. Джин Голуб Модель основана на аннотации языка Маркова + Операторы коммуникации Операторы: - send (A, let), где A - что посылаем, let - куда посылаем

Из всех топологий коммуникаций Голуб особо выделяет четыре:

Линия и кольцо

p - число задач

$1 \leq \mu \leq p$ - число процессов

let - $\mu - 1$

right - $\mu + 1$

Решетка и тор

(μ, λ) - номер задачи

north $(\mu - 1, \lambda)$, south $(\mu + 1, \lambda)$, west $(\mu, \lambda - 1)$, east

$(\mu, \lambda + 1)$

- recv(A, let), где A - куда принимаем, let - от кого принимаем

При записи параллельного алгоритма каждой задаче должны быть поставлены в соответствии инструкции, которая она выполняет, и данные, над которыми она работает

По Голобу алгоритму предваряется инициализация

Инициализация - p число задач, μ или $\mu - 1$ - номер задачи, локальные данные

У Голоба нет записи глобальных алгоритмов

Механизм распределения начальных данных между задачами:

Таблица 1.2. Распределение матрицы A между задачами алгоритма

	столбцовое хранение	строчное хранение
линейное распределение	$A(:, (\mu-1)r+1: \mu r)$	$A((\mu-1)r+1: \mu r, :)$
циклическое распределение	$A(:, \mu:p:n)$	$A(\mu:p:n, :)$

Пусть необходимо распределить n элементов вектора $a(1:n)$ между p задачами.

1. Линейное распределение

Есть массив $a(1 : n)$, $r = \frac{n}{p}$

В ходу инициализации $\mu - a((\mu - 1), r + 1)$

```

if mu = 1 then
  for i = 2 : p
    send(a((i-1)r+1:in), i)
  endfor
else
  recv(a(1:n),1)
endif

```

2. Циклическое распределение

$\mu - a(\mu : p : n)$

$$A((n-1)^{r+1} : y^r, \dots)$$

$$A \mid \sigma_j(\lambda-1) \wedge \vdash 1 : \lambda$$

Аннеке

$$A(y, p, n, \epsilon)$$

До 40 400 400 400

A diagram of a cylinder with vertical lines representing paths. The top is labeled with $\lambda + p$, $\lambda + 2p$, etc. The center has a large A . The bottom is labeled $A / (\lambda, p, h)$.

De xpaen
ro epress

Буклиссне

4

Линейное (блочное) распределение

Массив делится на непрерывные блоки, каждый процесс получает свой кусок:

Данные: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

4 задачи:

Задача 0: [0, 1, 2]

Задача 1: [3, 4, 5]

Задача 2: [6, 7, 8]

Задача 3: [9, 10, 11]

Каждая задача получает блок размером N / P , где N — размер данных, P — число задач.

Циклическое (Round-Robin) распределение

Элементы раздаются по одному поочерёдно, как карты в колоде:

Данные: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

4 задачи:

Задача 0: [0, 4, 8]

Задача 1: [1, 5, 9]

Задача 2: [2, 6, 10]

Задача 3: [3, 7, 11]



5. !Сети как модели вычислительных процессов (определение, свойства).!

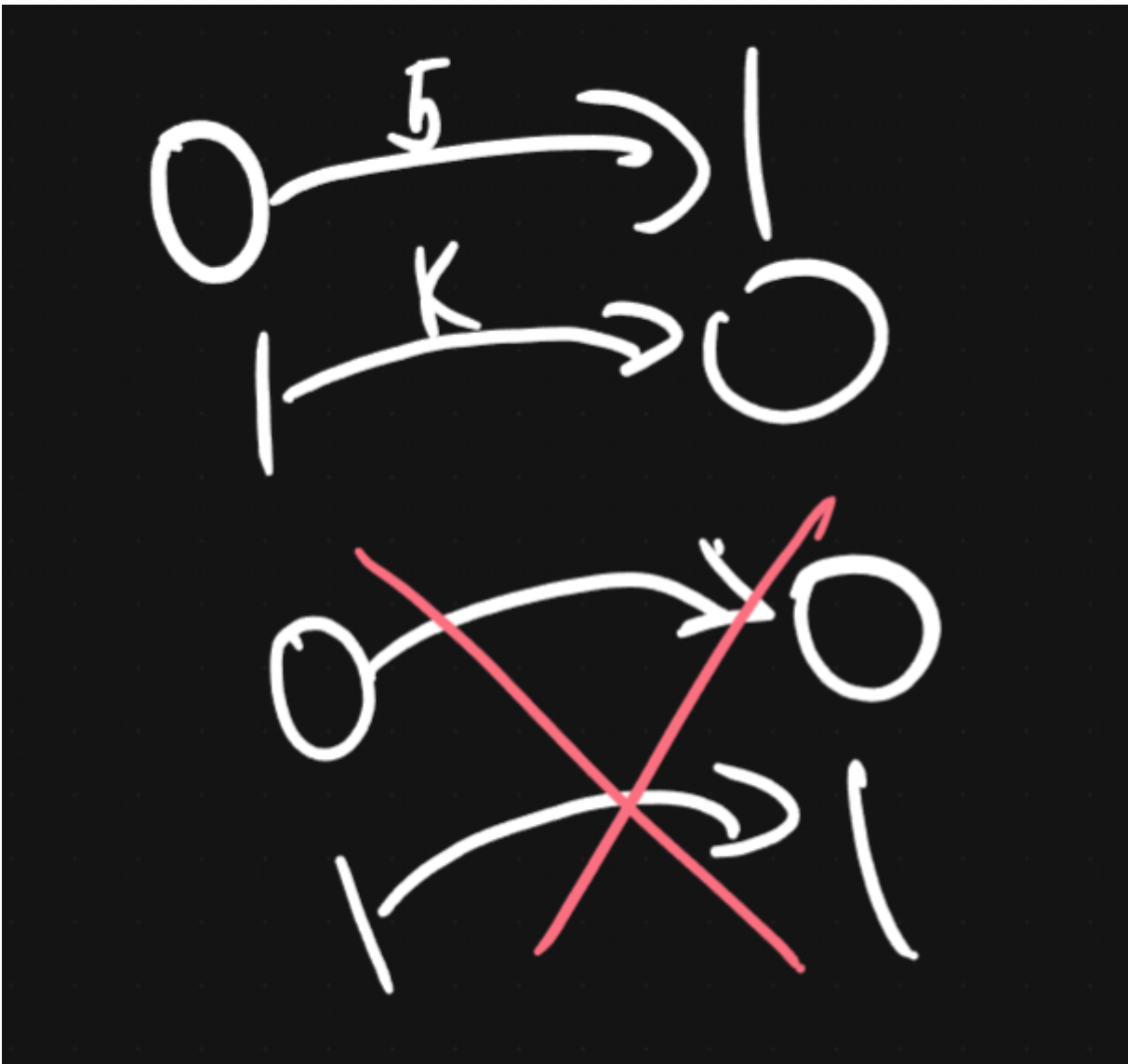
Вычислительный процесс - это действия по алгоритму на определенной вычислительной системе некоторое время

Сетью назовем - $N = (P, T, F)$

P - непустое конечное множество мест (Круги на графе)

T - непустое конечное множество переходов (Отрезки на графе)

F - функция инцидентности, имеет область определения: $F : P \times T \cup T \times P \rightarrow N$



Свойства сети:

1. Переходы и места разные множества $P \cap T = \emptyset$

Множества мест и переходов не пересекаются, являясь разными множествами по своей сути. Переходы обычно связывают с действиями вычислительного процесса, а места с условиями производства этих действий.

2. $\forall p \in P \exists t \in T$ либо $F(p, t) \neq 0$ либо $F(t, p) \neq 0$

Любое место сети связано хотя бы с одним переходом. Либо дуга ведет из такого перехода в данное место, либо из места в такой переход

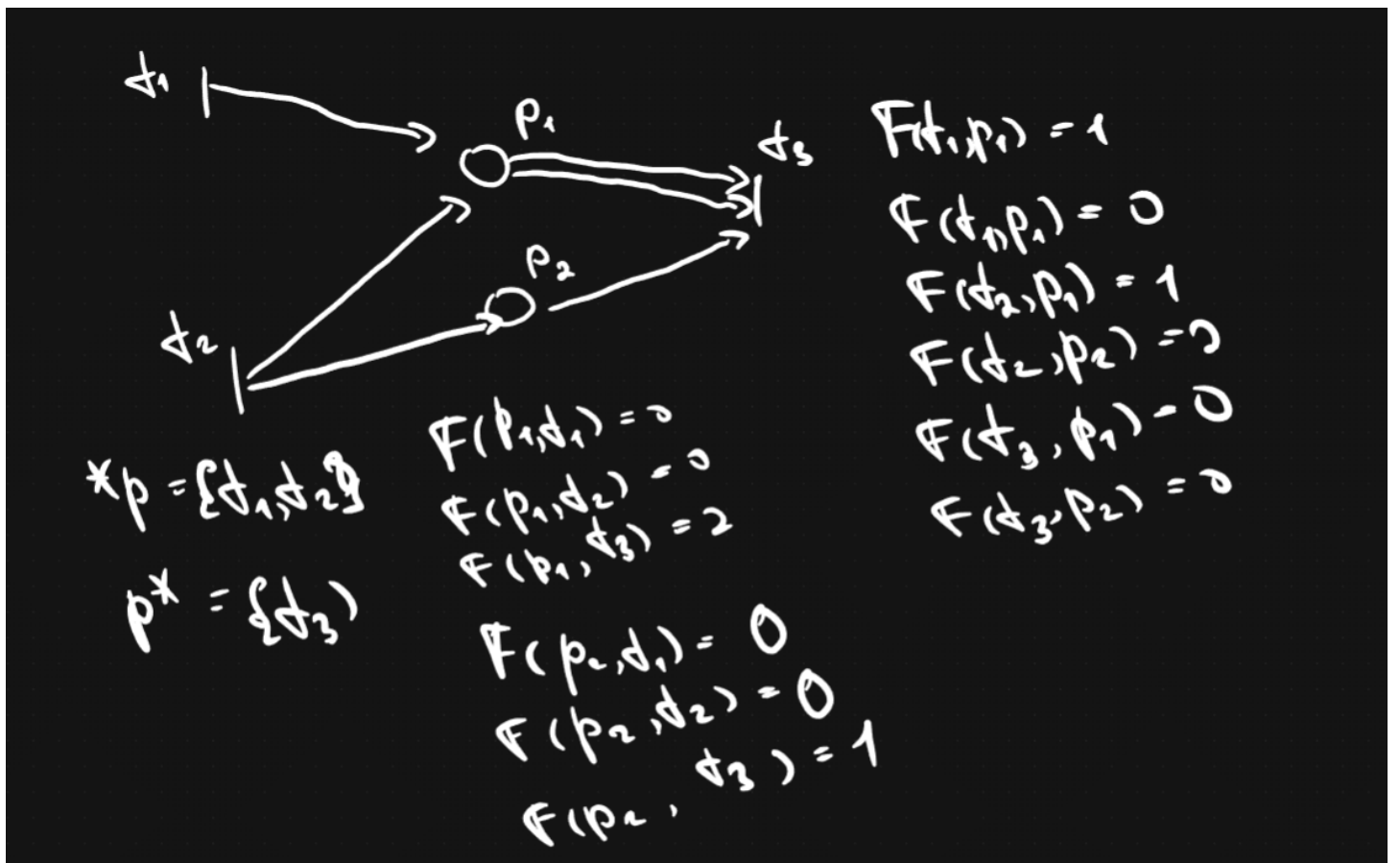
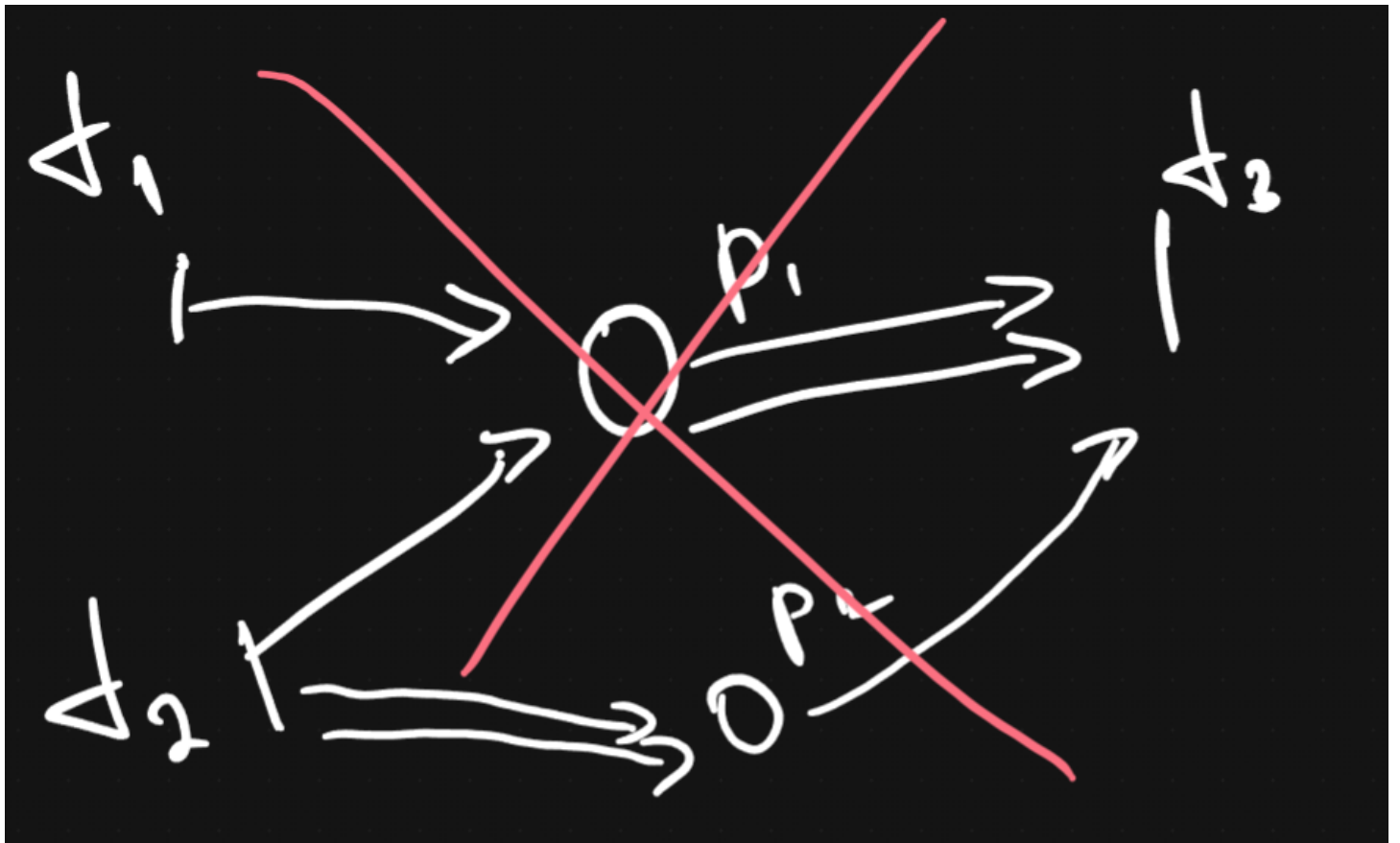
3. $\forall t \in T \exists p \in P$ либо $F(t, p) \neq 0$ либо $F(p, t) \neq 0$

Аналогично для переходов, Любой переход сети связан хотя бы с одним местом. Либо из такого места ведет дуга в данный переход, либо из перехода в такое место.

4. $\forall p_1, p_2 \in P \quad (*p_1 = *p_2) \text{ и } (p*_1 = p*_2) \rightarrow p_1 = p_2$

где $*p$ — множество входных переходов p , $p*$ — множество выходных переходов p

Согласно последнему свойству сети отрицается необходимость в существовании двух разных мест одинаково инцидентных одними и теми же переходам. Причем для переходов аналогичное свойство не формулируется.



6. !Сети Петри как модели вычислительных процессов (определения, формализм).!

Сеть Петри $PN = (P, T, F, M)$

M - множество $(M : P \rightarrow \mathbb{N})$ - разметка сети.

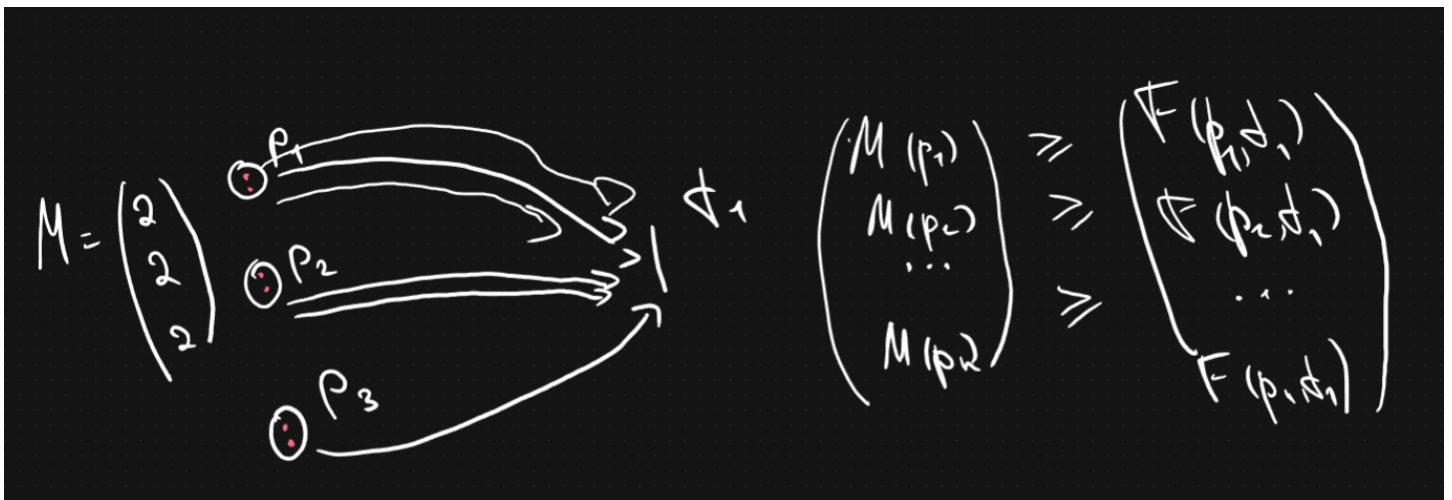
$$M = \begin{pmatrix} M(p_1) \\ M(p_2) \\ \dots \\ M(p_n) \end{pmatrix}$$

разметка меняется после срабатывания переходов.

Разметка есть функция, областью определения которой является множество мест, а областью значений – множество натуральных чисел с нулем. Графически значения разметки изображаются фишками внутри мест.

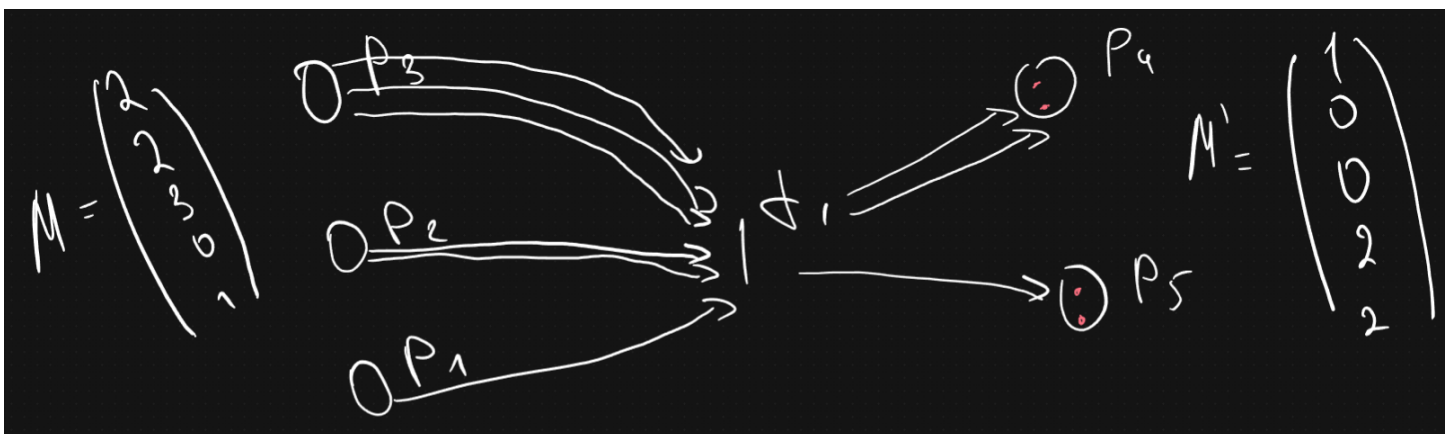
Условия срабатывания переходов: 1. Переход t - может (но не должен) сработать, если $\forall p \in {}^*t M(p) \geq F(p, t)$ в векторном виде $M \geq {}^*F(t)$

где t^* - множество входных мест перехода t

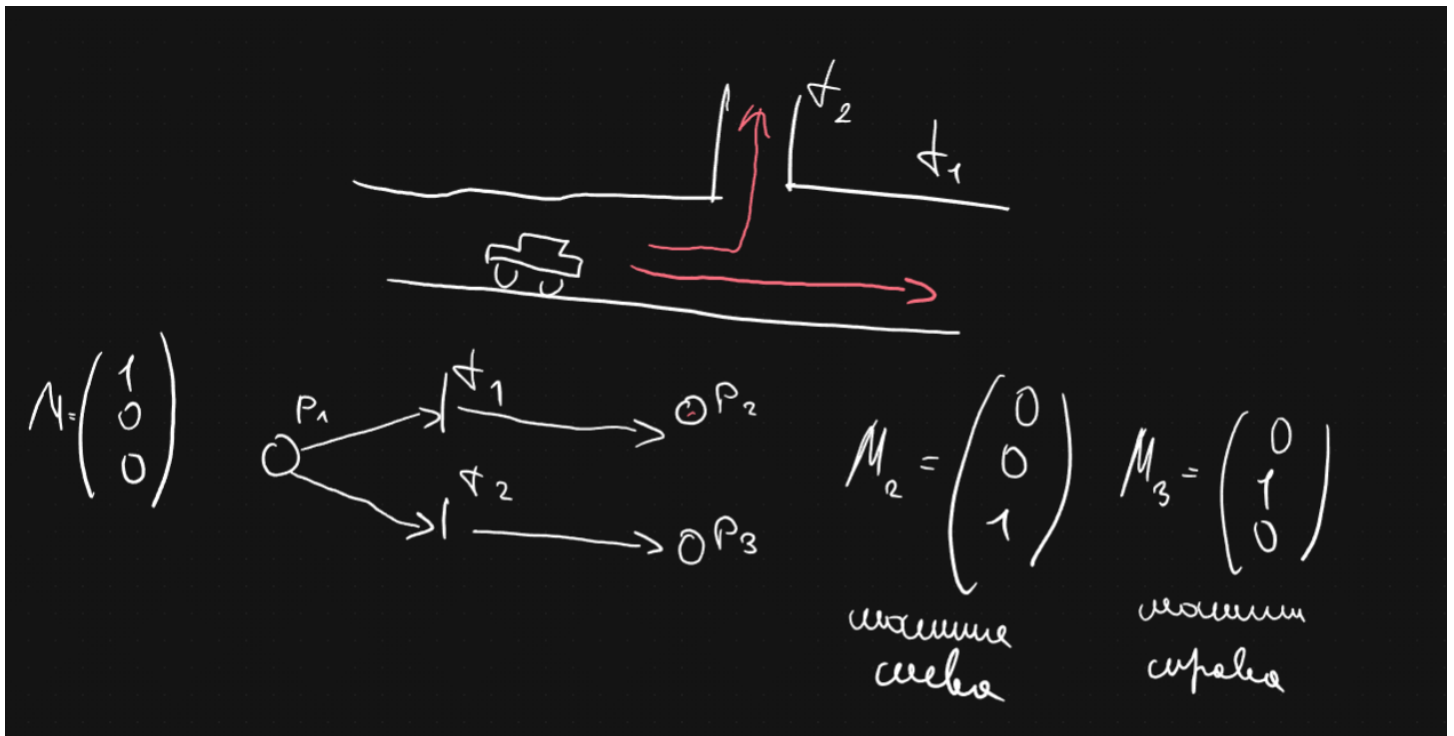


1. Допустим мог сработать и сработал, то меняется разметка $\forall p \in P M'(p) = M(p) - F(p, t) + F(t, p)$, в векторном виде $M' = M - {}^*F(t) + F^*(T)$

$$\begin{pmatrix} M(p_1) \\ M(p_2) \\ \dots \\ M(p_n) \end{pmatrix} = \begin{pmatrix} M(p_1) \\ M(p_2) \\ \dots \\ M(p_n) \end{pmatrix} - \begin{pmatrix} F(p_1, t) \\ F(p_2, t) \\ \dots \\ F(p_n, t) \end{pmatrix} + \begin{pmatrix} F(t, p_1) \\ F(t, p_2) \\ \dots \\ F(t, p_n) \end{pmatrix}$$



Пример:



Закона сохранения фишек **НЕТ!**

Работа всей сети Петри описывается:

1. Множество сработавших переходов $\tau = \{t_1, t_2 \dots\}$
2. Множество достижимых разметок $R_0 = (PN, M_0)$

$$M_0[t_1 > M_1[t_2 > M_2 \dots M_N$$

$M_0[t_1 > M_1$ - бинарное отношение непосредственно на множестве разметок

M_0, M_1 вступают в бинарное отношение непосредственного следования, если:

$$(M_0 \rightarrow *F(t_1)) \text{ и } (M_1 = M_0 - *F(t_1) + F^*(t_1))$$

АВТОМАТИЧЕСКОЕ РАСПАРАЛЛЕЛИВАНИЕ

7. Автоматическое распараллеливание ациклических фрагментов последовательных программ (построение стандартного графа и графа зависимостей).

Любой указанный фрагмент допускает запись через операторы всего двух видов: - Присваивание - Условные переходы

Этап I - Построение стандартного графа

Будем различать два типа вершин стандартного графа:

1. **Преобразователь** - каждый соответствует одному или нескольким последовательно расположенных операторов присваивания, изображается на графе **прямоугольником**
2. **Распознаватель** - каждый соответствует строго одному оператору условного перехода, изображается на графе **окружностью**

Дуги на графе задаются бинарным отношением непосредственного следования на множестве операторов

Отметим, что из преобразователя выходит не более одной дуги

Из распознавателя не более двух, при этом дугу соответствующей истинности условия распознавателя маркируем 1, ложное 0

Вершину в которую не входит ни одна дуга именуем **входной** вершиной стандартного графа

Вершину из которой не выходит не одна дуга именуем **выходной** вершиной стандартного графа

Пример:

A: $\text{begin}(x, y)$

B: $l = x$

C: $k = y$

D: $v = c + y$

E: $z = c + x$

P: Если $x > y$, то на F, иначе G

F: $l = x; l = y$

G: $\text{переменная}(\min(z, v), \max(z, v))$

H: $\text{сказать}(l, k)$



Этап II - Построение графа зависимостей

1. Информационная зависимость - вершина B информационно зависит от A, если {A и B находятся на одном пути стандартного графа из входной вершины в выходную (при чем A предшествует B)} и хотя бы один оператор из B использует значение переменной формируем хотя бы одним оператором из A. При этом отсутствует расположенная на одном пути, такая что меняется значение

При построение графа зависимости в него переносятся вершины стандартного графа без изменений

2. Логическая зависимость - вершина B логически зависит от A, если {...} и существует другой путь на стандартном графе из входной вершины в выходную. Совпадающий с первым до вершины A включительно, далее отличающуюся от него и не содержащую B.
3. Конкуренционная зависимость - вершина B конкуренционно зависит от вершины A, если {...} и в каждой из вершин есть оператор присваивания меняющий значение некой общей переменной.

Отныне не различаем тип зависимости. Завершаем построение графа зависимости нанесение на него транзитивного замыкания зависимостей построенных к этому моменту.



$\rightarrow$ инф. зависимость
 $\xrightarrow{\wedge}$ лог. зав.
 $\xrightarrow{k}$ лог. зав.

$----->$ транз. замыкание
 $Dep(A, B)$

8. Автоматическое распараллеливание ациклических фрагментов последовательных программ (построение ЯПФ и параллельного алгоритма).

Этап III - Построение ярусно параллельной формы

1. A
2. B; C; D; E; F
3. F; G
4. H

Алгоритм построения ЯПФ:

Шаг 1. Заполнение первого яруса ЯПФ

В первый ярус помещаются $A \nexists BDep(B, A)$

Шаг 2. Пусть k ярусов заполнено, если $\forall A \exists_{i \leq k} \exists_{j \leq n_k} A = A_{ij}$, то ЯПФ построен, иначе шаг 3.

Шаг 3. В $k + 1$ ярус помещаются такие вершины $A \quad Dep(B, A) \implies \exists_{i \leq k} \exists_{j \leq n_k} A_{ij} = B$, шаг 2.

В итоге на каждом ярусе окажутся исключительно независимые вершины, их операторы могут **выполняться одновременно**.

Этап IV - Построение параллельного алгоритма

Метод построения параллельного алгоритма: Пусть известно p - число задач и $1 \leq \mu \leq p$ - номер задачи

Шаг 1. Распределение вершин первого яруса ЯПФ между задачами

$$\mu \sim A_{1,\mu}; A_{1,\mu+p}; A_{1,\mu+2p} \dots$$

Шаг 2. Пусть k ярусов распределено

Если ЯПФ закончилось то алгоритм построен, иначе переходим на шаг 3.

Шаг 3. Распределение $k + 1$ яруса ЯПФ по задачам параллельного алгоритма

$$\mu \sim c(A_{k+1,\mu}); A_{k+1,\mu}; c(A_{k+1,\mu+p}); A_{k+1,\mu+p}; c(A_{k+1,\mu+2p}); A_{k+1,\mu+2p} \dots$$

Если функция $c(A)$ - истина, то мы исполняем операторы вершины за ней следующей.

Если ложь, тогда операторы следующей вершины не выполняются, а проверяются значения функции следующей за той вершиной.

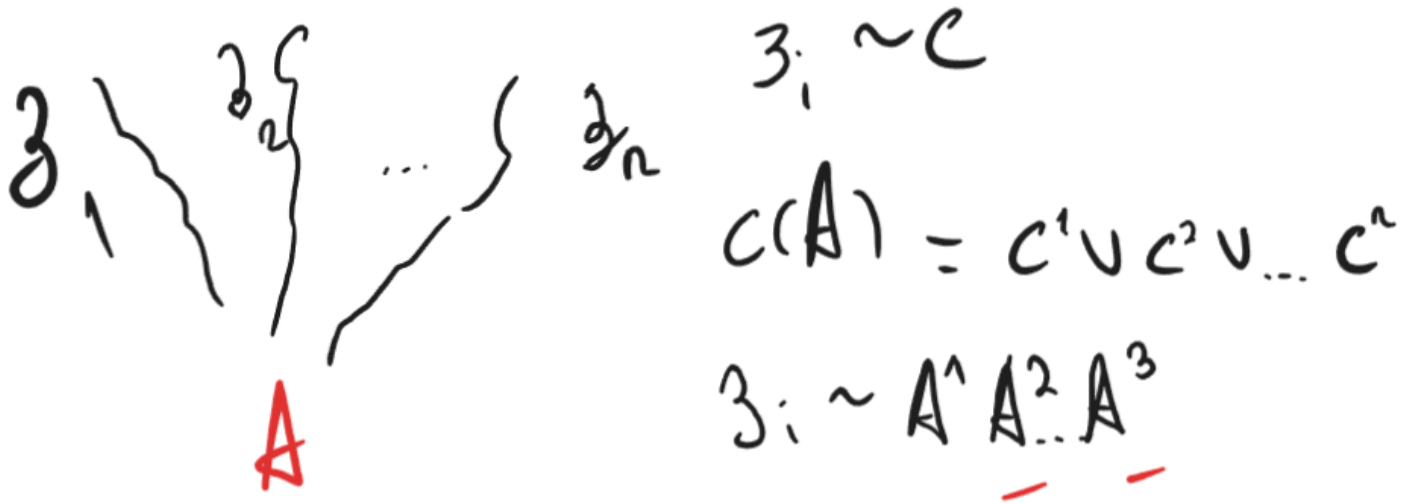
Если пока не определено, то вычислительный процесс прерывается, а μ — задача простаивает, пока значение $c(A)$ не станет определенным.

Переход на шаг 2.

9. Автоматическое распараллеливание ациклических фрагментов последовательных программ (формализм определения функции «с»).

Формализм определения функции $c(A)$ Если далее говорится о пути, то имеется ввиду стандартный граф. Говоря о зависимости имеется ввиду граф зависимостей. Пусть через вершину A проходит n путей из входной вершины в выходную — $\xi^1, \xi^2, \xi^3, \dots, \xi^n$, тогда каждой ставим в соответствие $c^1, c^2, c^3, \dots, c^n$, принимающие значения истинно, если вычислительный процесс идущий по этому пути дошел до вершины A . Ложь, если заведомо известно, что пошел или пойдет другим путем, не содержащим A . c^i — неопределенно, если не одно из предыдущих утверждений пока не верно. Тогда $c(A)$ есть дизъюнкция. Тогда $c(A) = c^1 \cup c^2 \cup c^3 \dots \cup c^n$.

Выделим на данном пути, от которого A зависит. Тогда $c(A_k)$, где $1 \leq k \leq K \Rightarrow c^i = c(A_1) \cap c(A_2) \cap \dots \cap c(A_K)$.



$$(A_k) \begin{cases} c(A_k) - \text{Если } A_k - \text{преобразователь} \\ c(A_k = 1) - \text{Если } A_k - \text{распознаватель, и для реализации пути } \xi_i \text{ должно стать истинной} \\ c(A_k = 0) - \text{Если для реализации пути } \xi_i \text{ условие распознавателя } A_k - \text{ложно} \end{cases}$$

10. Распараллеливание циклических фрагментов программ (пространство итераций, ускорение, метод пирамид).

```
(*)
for i_1 = 1:n_1
  for i_2 = 1:n_2
    ...
    for i_k = 1:n_k
      T(i_1, i_2, ..., i_k)
    end
    ...
  end
end
```

$$I = \{(i_1, i_2, \dots, i_k) : 1 \leq i_g \leq n_g, 1 \leq g \leq k\}$$

I - пространство итераций

$(i_1, i_2, \dots, i_k)$ - вектор из пространства итераций

1. Отношение следования

Развернем конструкцию (*) следующим образом:

$$(\wedge) T(1,1, \dots, 1); T(1,1, \dots, 2); T(1,1, \dots, n_k); T(1,1, \dots, 2,1); \dots T(n_1, n_2, \dots, n_k)$$

Теперь строим стандартный граф. Ищем транзитивное замыкание отношения непосредственного следования, назовем его **отношения следования** на множестве вершин.

Между пространством итераций определенной (*) и телами из развернутой ациклической конструкции ($\wedge$), очевидно, существует взаимно однозначное отображение (биекция).

Тогда будем говорить, что вектор i из пространства итераций (связанный с телом T) следует за вектором i' (связанный с телом T'), если любая вершина из тела T конструкции ($\wedge$) следует за любой вершиной из тела T' .

2. Отношение зависимость

По развернутой ациклической конструкции ($\wedge$) строим граф зависимость.

Далее будем говорить, что вектор i (относящийся к T) зависит от i' (относящийся к T'), если хотя бы одна вершина из T зависит хотя бы от одной из T' .

Целью математической части лекции является поиск покрытия пространства итераций $\{I_q\}_{q=\overline{1,Q}}$

Если $I = \bigcup_{q=1}^Q I_q$, то $\{I_q\}_{q=\overline{1,Q}}$

При покрытии $I_1 \cap I_q \neq \emptyset$

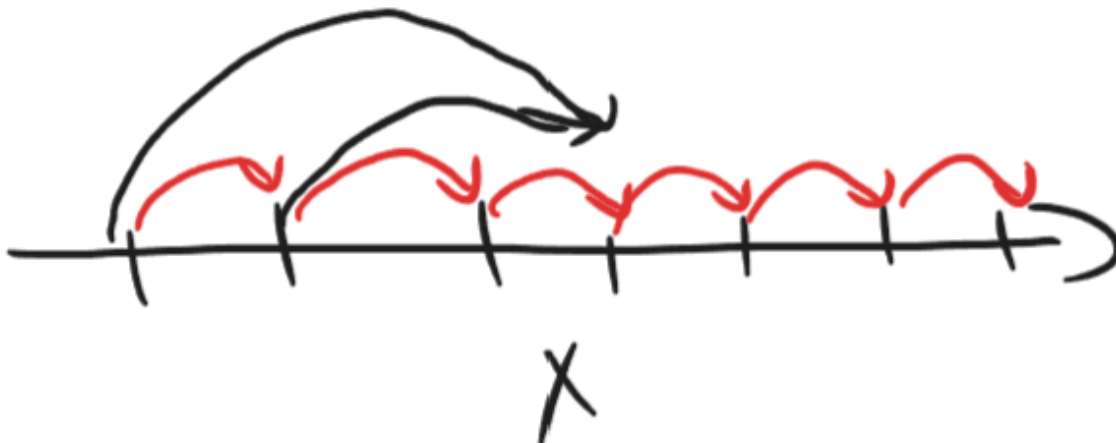
Наша цель найти покрытие, такое что любое его подмножество содержит исключительно независимые векторы.

В одном подмножестве покрытия итерации связанный с его векторами можно исполнять одновременно.

$$S = \frac{\prod_{j=1}^k n_j}{Q} = \frac{I_{\text{посл}}}{I_{\text{паралл}}} \rightarrow Max \quad - \quad \text{ускорение (speedup)}$$

Пример (?):

```
for i = 1:10
    x(i) = f(x(i-1))
end
```



Для дальнейшего построения параллельного алгоритма необходимо соблюдения следующих условий:

1. Параметры цикла внутри его тела не должны изменяться
2. В теле цикла отсутствует переход за циклическую конструкцию
3. Параметры цикла в теле цикла участвуют только в линейных операциях

Пункт 2.2.1 - Метод пирамид

Шаг1. По циклической конструкции (*) строится пространство итераций на нём вводятся отношения следования и зависимость.

На пространстве итераций выбираются вектора каждый из которых не зависит не от какого другого вектора этого пространства, назовем такие вектора **результатирующими**.

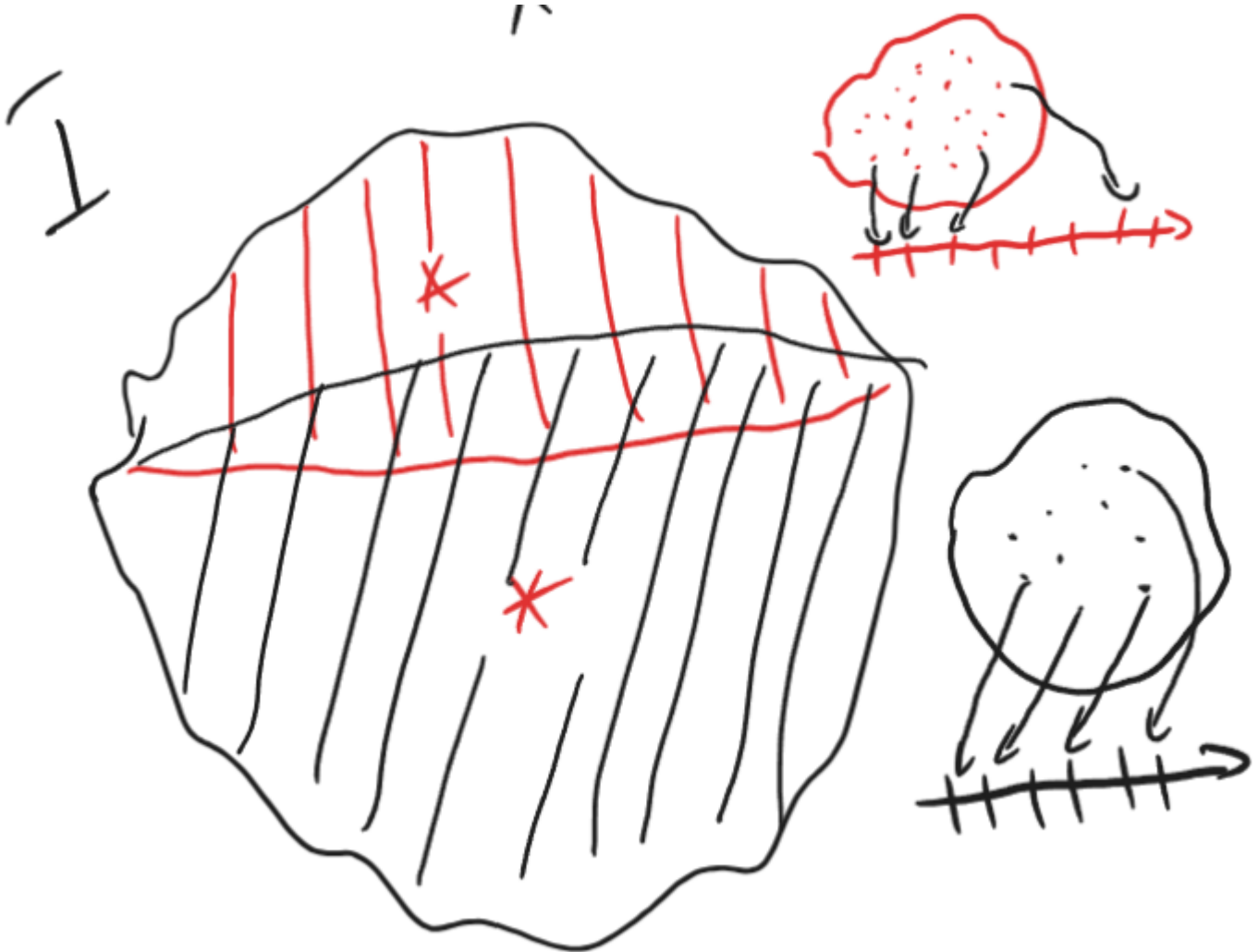
Шаг2. Каждому такому вектору поставим во взаимное однозначное соответствие (биекция) одну задачу параллельного алгоритма.

Кроме указанного вектора отнесем к ней также все вектора пространства итераций от которых данные **результатирующие** зависят.

Шаг3. Внутри каждой задачки упорядочим отнесенные к ней вектора в соответствие с отношением следования.

Алгоритм построен.

В отличие от других методов построения **Метод Пирамид** не характеризуется коммуникацией.



Пример Метода пирамид

Последовательный алгоритм:

```

for i = 1:I
  for j = 1:J
    u(i,j) = f(u(i-1,j),u(i-1,j-1)mu(i-1,j+1))
  endfor
endfor

```

Параллельный алгоритм:

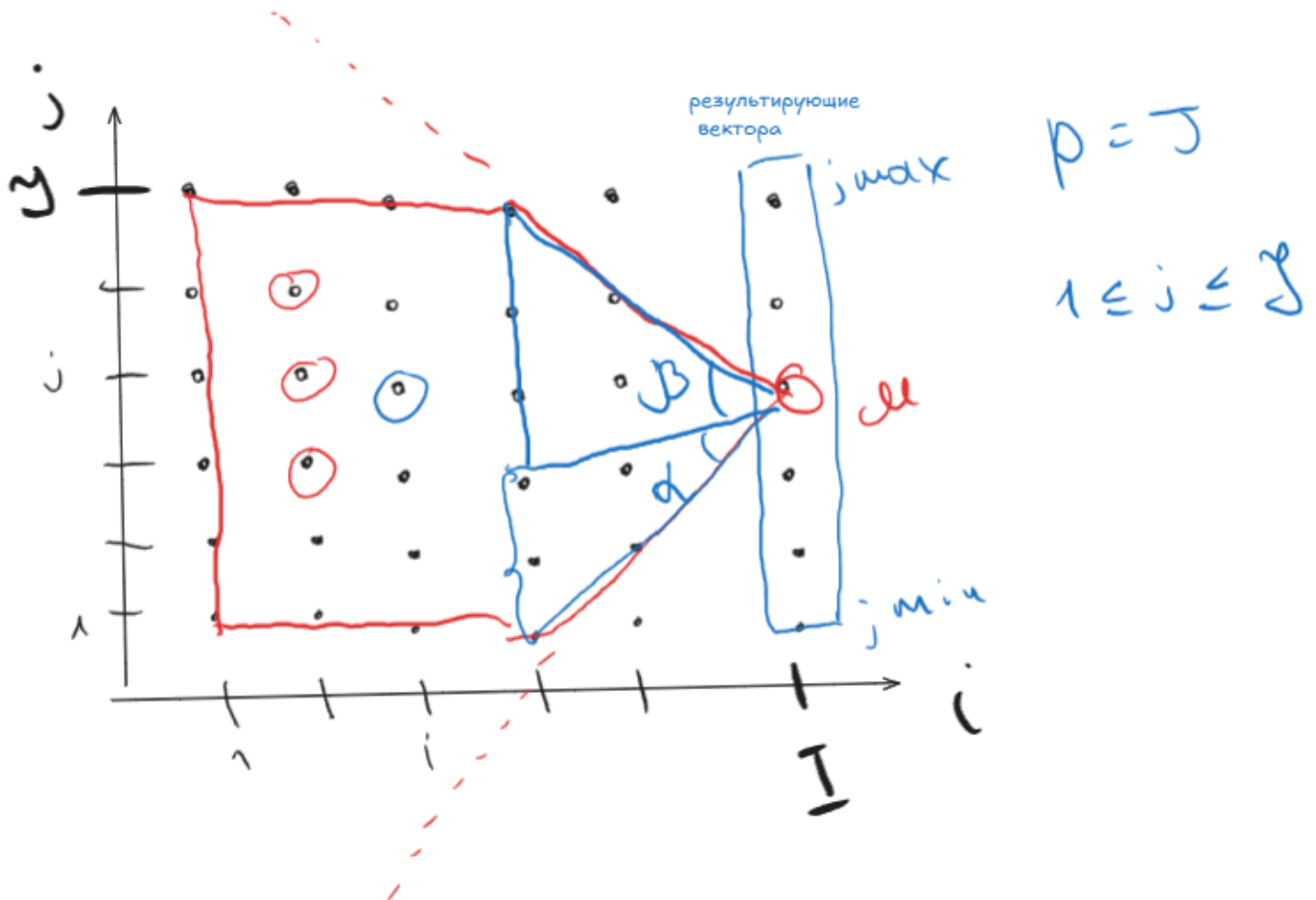
```

for i = 1:I
  for j = max{1,mu-(I-i)tg(alpha)} : min{J,mu+(I-i)tg(beta)}
    u(i,j) = f(u(i-1,j),u(i-1,j-1)mu(i-1,j+1))
  endfor
endfor

```

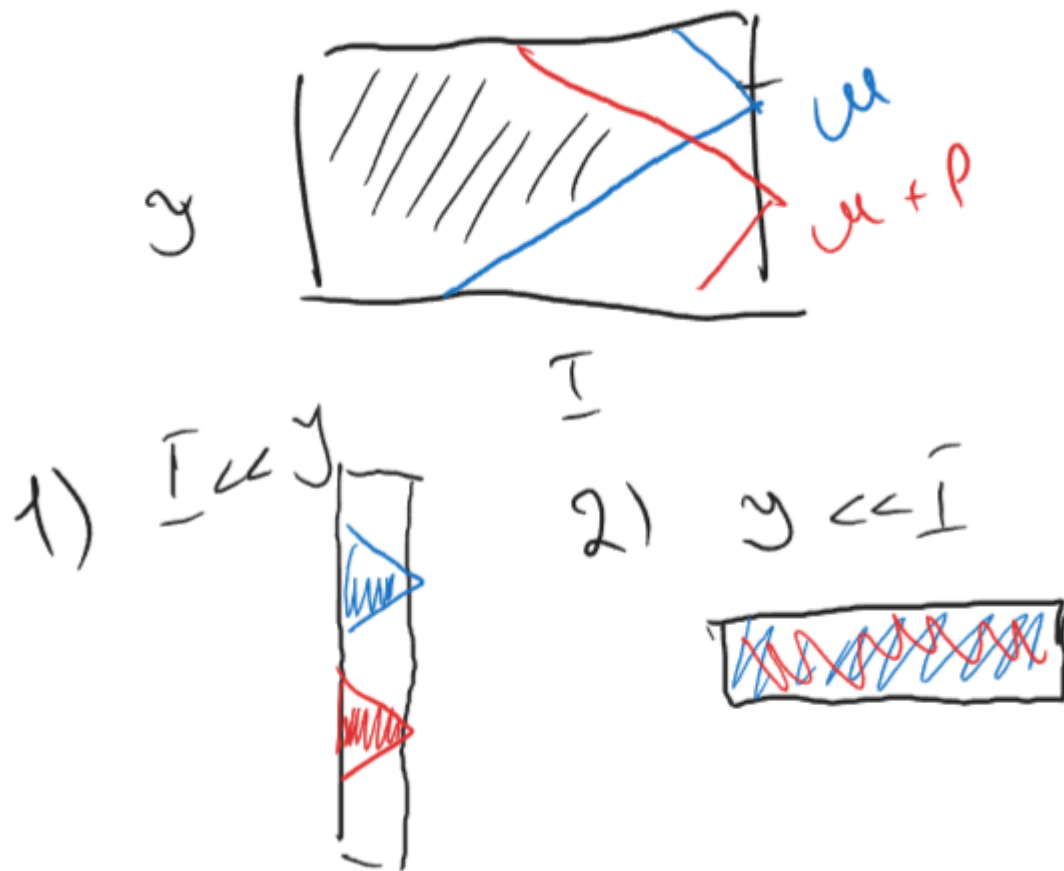
$\mu - (I - i)tg(\alpha) - j \min$

$\mu + (I - i)tg(\beta) - j \max$



Замечания

1. Метод имеет смысл применять в 1) случае, а не 2):



2. На практике задачи с близкими номерами μ принято объединять



3. В отличие от остальных методов в данном допускается использование параметров цикла внутри цикла в нелинейных выражениях

11. Распараллеливание циклических фрагментов программ (пространство итераций, ускорение, метод параллелепипедов).

В отличие от всех остальных методов позволяет работать с простыми циклами

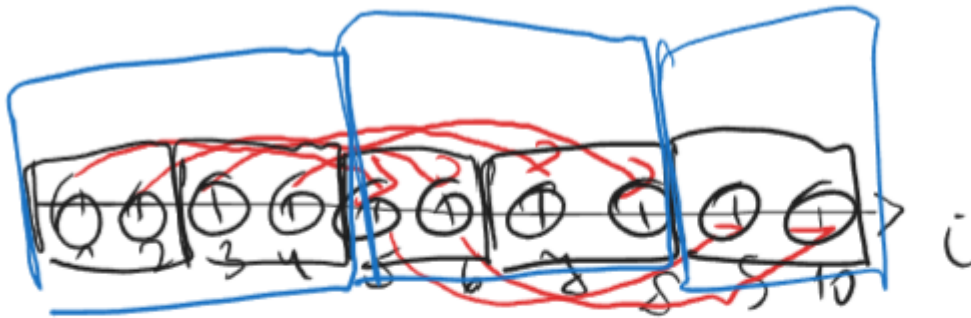
Последовательный алгоритм:

```

for i = 1:10
    x(i) = f(x(i-4))
endfor

```

Параллельный алгоритм:



$$p = 2 \quad 1 \leq \mu \leq 2$$

$$\begin{array}{cccc}
 \mu = 1 & \mu = 2 & \mu = 3 & \mu = 4 \\
 1 & 2 & 3 & 4 \\
 5 & 6 & 7 & 8 \\
 9 & 10 & &
 \end{array}$$

```

for i = mu:4:10
    x(i) = f(x(i-4))
endfor

```

$$S = \frac{10}{3}$$

```

for i_1 = 1:n_1
    for i_2 = 1:n_2
        ...
        for i_k = 1:n_k
            T(i_1, i_2, ..., i_k)
        endfor
        ...
    endfor
endfor

```

Шаг 1. Строим пространство итераций, наносим на него отношение следования и зависимости.

Шаг2. На пространстве итераций ищем различные покрытия, каждый из которых представляет из себя набор k -мерных параллелепипедов.

Шаг3. Для каждого из этих покрытий выбираем параллелепипед максимального объема и на его ребрах строим новую систему координат.

Количество задач параллельного алгоритма будет равным числу векторов в выбранном промежутке.

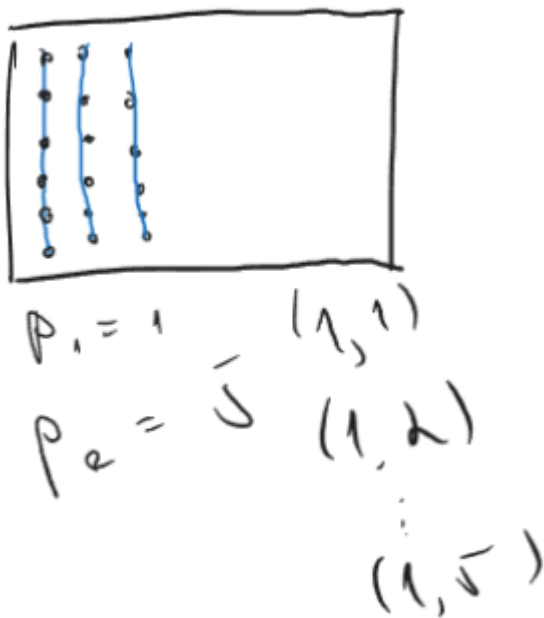
Если длины сторон параллелепипеда равны $p_1, p_2, \dots, p_k$ (где p_i есть разность координат последнего и первого вектора по i направлению + 1), то для прямоугольного параллелепипеда общее число задач есть: $p = \prod_{i=1}^k p_i$

За номер задачи примем координату вектора в выбранном параллелепипеде в новой система координат

$(g_1, g_2, \dots, g_k)$

Может случится, что размерность выбранного параллелепипеда окажется меньше пространства итераций, в этом случае одна или несколько p будут приняты за 1

А в номере задачи соответствующая ему координата также будет принята за единицу



Шаг4. Отнесем к ведению каждой задачи не более одного вектора из каждого подмножества выбранного покрытия

Внутри каждой задачи совокупность отнесенных к ней векторов расположены в соответствии с отношением следования, таким образом:

```
for i_1 = g_1:p_1:n_1
  for i_2 = g_2:p_2:n_2
    ...
    for i_k = g_k:p_k:n_k
      T(i_1,i_2,...,i_k)
      + коммуникация
    endfor
    ...
  endfor
endfor
```

12. Распараллеливание циклических фрагментов программ (синтетический алгоритм, объединяющий метод параллелепипедов и пирамид).

—нейропояснения—

Синтетический алгоритм — это объединение двух методов:

- Метод параллелепипедов — делит пространство итераций равномерно между задачами (шаг p_1, p_2). Каждая задача берёт свой "кусок" сетки.
- Метод пирамид — обрабатывает зависимости между итерациями. Когда нужные данные считает другая задача — передаём через `recv/send`.

Вместе они дают алгоритм который и равномерно распределяет работу и корректно обрабатывает зависимости через коммуникацию между задачами.

В отличие от чистого метода пирамид — здесь коммуникация есть, но задачи более равномерно загружены.

Параллелепипед говорит **кто что считает**, пирамида говорит **кто кому передаёт данные** на границах своего куска.

—конецнейропояснений—

Двумерный пример:

```
for i=1:7
  for j=1:3
    x(i,j) = f(x(i-2,j-1))
  endfor
endfor
```

Параллельный:

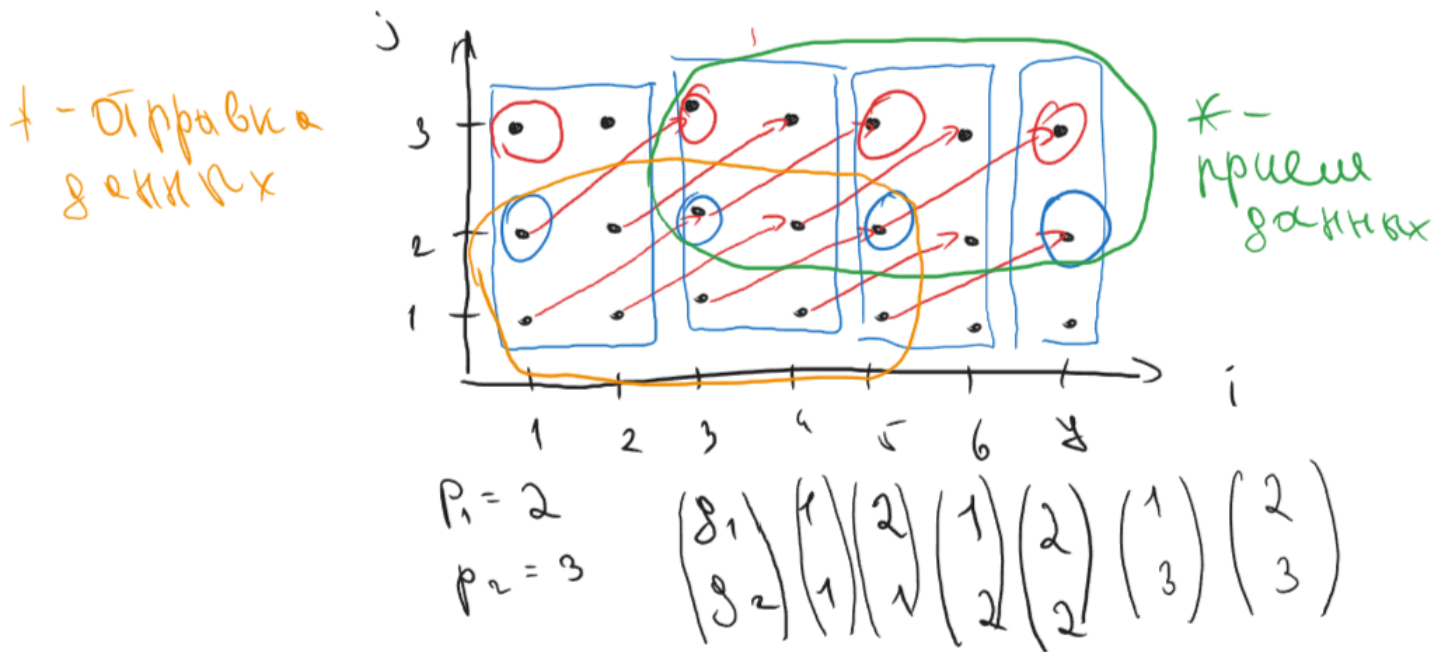
```
for i=g_1:p_1:7
  for j=g_2:p_2:3
    if (i>2) and (j>1) then
      recv(x(i-2,j-1), (g_1,g_2-1))

      x(i,j) = f(x(i-2,j-1))

      if(i<6) and (j<3) then
        send(x(i,j), (g_1,g_2+1))
      endfor
    endfor
  endfor
```

Для задачи в кружочке:

```
for i=1:2:7 ⇒ i = 1,3,5,7
  for j=3:3:3 ⇒ j = 3
    T(i,j)
  endfor
endfor
```



Как в прошлом случае количество задач найденных таким образом много больше числа используемых устройств, поэтому задачи объединяют.

ВЕКТОРНЫЕ АЛГОРИТМЫ

13. !Основные операции с векторами и матрицами. Векторные алгоритмы умножения матриц.!

Пункт 3.1.1 Основные векторные операции

Операция с вектором (аппаратно)

1. $z = x + y$, где $z, x, y \in R^{n \times 1}$ - сложение
2. $z = \alpha x$, где $\alpha \in R$ - умножение вектора на скаляр
3. $z = x^T y$ - скалярное произведение
4. $z = x * y$ - покомпонентное произведение
5. $z = \alpha x + y$ - $\alpha x + y$ (scalar αx plus y)

Операции с матрицами

1. $z = Ax + y$, где $A \in R^{n \times m}$ $z, y \in R^{n \times 1}$ $x \in R^{m \times 1}$ - $Ax + y$ (matrix A times x plus y)

$$\begin{pmatrix} | \\ | \end{pmatrix} = \begin{pmatrix} | \\ | \end{pmatrix} \begin{pmatrix} | \\ | \end{pmatrix} + \begin{pmatrix} | \\ | \end{pmatrix}$$

z A x y



$z \rightarrow y$

(или с помощью)

сaxpy через скалярное произведение

```
for i = 1:n
    y(i) = A(i,:)x + y(i)
endfor
```

План Б. Выражение сaxpy через saxpy

$$\begin{pmatrix} | \\ | \end{pmatrix} = \begin{pmatrix} | \\ | \end{pmatrix} \begin{pmatrix} | \\ | \end{pmatrix} + \begin{pmatrix} | \\ | \end{pmatrix} = \begin{pmatrix} | \\ | \end{pmatrix} x(1) + \begin{pmatrix} | \\ | \end{pmatrix} x(2) + \dots + \begin{pmatrix} | \\ | \end{pmatrix} x(m) + \begin{pmatrix} | \\ | \end{pmatrix}$$

z A x y $A(:,1)$ $A(:,2)$ $A(:,m)$ y

```
for j = 1:m
    y = y + A(:,j)x(j)
endfor
```


$$2. A = A + xy^T, \text{ где } A \in R^{n \times m} \quad x \in R^{n \times 1} \quad y \in R^{m \times 1}$$

Вся операция вместе - называется модификация матрицы внешним произведением

```
for i=1:n
    A(i,:) = A(i,:) + x(i)y^T - строчное saxpy
endfor

for j=1:m
    A(:,j) = A(:,j) + xy(j) - столбцовое saxpy
endfor
```

Пункт 3.1.2 Векторные алгоритмы умножения матриц

Умножение по определению:

```
for i=1:n
    for j=1:n
        for k=1:n
            c(i,j) = A(i,k)B(k,j)
        endfor
    endfor
endfor
```

Цикл k преобразуем в скалярное произведение:

```
for i=1:n
    for j=1:n
        c(i,j) = A(i,:)B(:,j)
    endfor
endfor
```

Цикл j убираем, получаем saxpy:

```
for i=1:n
    c(i,:) = A(i,:)B - строчное saxpy
endfor
```

$z = xA + y$, где $A \in R^{n \times m}$ $x \in R^{n \times 1}$ $y \in R^{m \times 1}$ - строчное saxpy

Название алгоритма	Векторная операция	Название операции	$c = A * B$ Доступ к памяти
<i>ijk</i>	скалярное произведение	строчное saxpy	смешанная
<i>kij</i>	строчное saxpy	модификация матрицы	по строкам

```
for k=1:n
    for i=1:n
        for j=1:n
            c(i,j) = c(i,j) + A(i,k)B(k,j)
        endfor
    endfor
endfor
```

Свернем j цикл (saxpy строчное):

```
for k=1:n
    for i=1:n
        c(i,:) = c(i,:) + A(i,k)B(k,:) - saxpy строчное
    endfor
endfor
```

Свернем i цикл (модификация матрицы):

```

for k=1:n
    c = c + A(:,k)B(k,:) - модификация матрицы
endfor

```

Блочный алгоритм:



A, B, C $\alpha = \frac{n}{N} \leftarrow$ блокная размер.



```

for i_b = 1:N
    i=1(i_b-1)alpha + 1 : i_b alpha
    for j_b = 1:N
        j=1(j_b-1)alpha + 1 : j_b alpha
        for k_b = 1:N
            k=1(k_b-1)alpha + 1 : k_b alpha
            c(i,j) = c(i,j) + A(i,k)B(k,j)
        endfor
    endfor
endfor

```

14. !Векторные алгоритмы решения СЛАУ с матрицей треугольного вида.!

Пункт 3.2.1 Решение СЛАУ с матрицами треугольного вида

$$Lx = b$$

$$\begin{pmatrix} l_{11} & 0 & 0 & \dots & 0 \\ l_{21} & l_{22} & 0 & \dots & 0 \\ l_{31} & l_{32} & l_{33} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & l_{n3} & \dots & l_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \dots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \dots \\ b_n \end{pmatrix}$$

$$x_1 = \frac{b_1}{l_{11}}$$

$$x_2 = \frac{b_2 - l_{21}x_1}{l_{22}}$$

$$x_3 = \frac{b_3 - l_{31}x_1 - l_{32}x_2}{l_{33}}$$

$$x_i = \left(b_i - \sum_{j=1}^{i-1} \frac{l_{ij}x_j}{l_{ii}} \right)$$

$L(i, 1 : i - 1)b(1 : i - 1)$ - скалярное произведение

```
for i=2:n
    b(i) = b(i) - L(i,1:i-1)b(1:i-1) / L(i,i)
endfor
```

Пусть L нижняя унитреугольная матриц

```
for i=2:n
    b(i) = b(i) - L(i,1:i-1)b(1:i-1) - строка
endfor
```

$$\begin{pmatrix} l_{11} & 0 & 0 & \dots & 0 \\ l_{21} & l_{22} & 0 & \dots & 0 \\ l_{31} & l_{32} & l_{33} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & l_{n3} & \dots & l_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \dots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \dots \\ b_n \end{pmatrix} - \begin{pmatrix} l_{21} \\ l_{31} \\ l_{41} \\ \dots \\ l_{n1} \end{pmatrix} - b'$$

$$x_1 = \frac{b_1}{l_{11}}$$

$$x_2 = \frac{b'_2}{l_{22}}$$

```
for j=1:n-1
    b(j) = b(j)/L(j,j)
    b(j+1:n) = b(j+1:n) - L(j+1:n,j)b(j) - saxpy (столбец)
endfor
b(n)=b(n)/L(n,n)
```

Смотрим на конкретный момент: $j=1$

Мы только что нашли x_1 :

$$b(1) = b(1) / L(1,1) \rightarrow b(1) \text{ теперь хранит } x_1$$

Теперь вопрос: куда x_1 входит в системе?

Выписываем уравнения:

$$\begin{aligned} \text{строка 2: } & l_{21} \cdot x_1 + l_{22} \cdot x_2 = b_2 \\ \text{строка 3: } & l_{31} \cdot x_1 + l_{32} \cdot x_2 + l_{33} \cdot x_3 = b_3 \\ \text{строка 4: } & l_{41} \cdot x_1 + l_{42} \cdot x_2 + l_{43} \cdot x_3 + l_{44} \cdot x_4 = b_4 \end{aligned}$$

x_1 входит в каждое уравнение ниже. Его вклад — это $l_{i1} \cdot x_1$ в строке i .

Мы хотим перенести это в правую часть:

$$\begin{aligned} \text{строка 2: } & l_{22} \cdot x_2 = b_2 - l_{21} \cdot x_1 \\ \text{строка 3: } & l_{32} \cdot x_2 + l_{33} \cdot x_3 = b_3 - l_{31} \cdot x_1 \\ \text{строка 4: } & l_{42} \cdot x_2 + l_{43} \cdot x_3 + l_{44} \cdot x_4 = b_4 - l_{41} \cdot x_1 \end{aligned}$$

То есть нужно выполнить три операции одновременно:

$$\begin{aligned} b(2) &\leftarrow b(2) - l_{21} \cdot x_1 \\ b(3) &\leftarrow b(3) - l_{31} \cdot x_1 \\ b(4) &\leftarrow b(4) - l_{41} \cdot x_1 \end{aligned}$$

Это и есть та строка кода

Смотрим что стоит на каждом месте:

$$b(2:4) = b(2:4) - L(2:4, 1) \cdot b(1)$$

- $b(2:4)$ — это $[b(2), b(3), b(4)]$
- $L(2:4, 1)$ — это первый столбец L , строки 2,3,4: $[l_{21}, l_{31}, l_{41}]$
- $b(1)$ — это x_1

Пусть L нижняя унитреугольная матриц

```
for j=1:n-1
    b(j+1:n) = b(j+1:n) - L(j+1:n,j)b(j) - saxpy (столбец)
endfor
```

$L = X = B$, где $L \in R^{n \times n}$ $X, B \in R^{n \times m}$

$$\frac{n}{N} = \alpha$$

$$L_{i_b, j_b} \in R^{\alpha \times \alpha}$$

$$B_{i_b}, X_{i_b} \in R^{\alpha \times m}$$

$$1 \leq i_b, j_b$$

$$n \begin{pmatrix} & & \\ & L_{i,j_b} & \\ & & \end{pmatrix} \begin{pmatrix} & \\ x_{i_b} & \\ & \end{pmatrix}_m = \begin{pmatrix} & \\ b_{i_b} & \\ & \end{pmatrix}_m n$$

```

for j_b = 1:N
    #решаем L_{j_b,j_b}X_{j_b} = B_{j_b} - СЛАУ
    for i_b = j_b+1:N
        B_{i_b} = B_{i_b} - L_{i_b,j_b}X_{j_b}
    endfor
endfor
L_{N,N}X_N=B_N

```

15. Векторные алгоритмы LU-разложения через модификацию матрицы внешним произведением.

Пункт 3.2.2 Алгоритм LU разложения через модификацию внешним произведением

$$\begin{pmatrix} 1 & 0 \\ -\frac{x_2}{x_1} & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_1 \\ 0 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 \\ -\frac{x_2}{x_1} & 1 \end{pmatrix} \text{ — матрица Гаусса}$$

$$\begin{pmatrix} 1 & 0 & \dots & 0 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & 1 & 0 & \dots & 0 \\ 0 & 0 & \dots & -\frac{x_{k+1}}{x_k} & 1 & \dots & 0 \\ \vdots & \vdots & & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & -\frac{x_n}{x_k} & 0 & \dots & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_k \\ x_{k+1} \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_k \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

$$\tau^{(k)} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ \frac{x_{k+1}}{x_k} \\ \vdots \\ \frac{x_n}{x_k} \end{pmatrix}$$

$$\tau_i^{(k)} = \begin{cases} 0, & \text{при } i \leq k \\ \frac{x_i}{x_k}, & \text{при } i > k \end{cases}$$

$$l_k = \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix}$$

$$\tau^{(k)} l_k^T = ???$$

$\tau^{(k)} l_k^T$ — внешнее произведение

Это матрица $n \times n$:

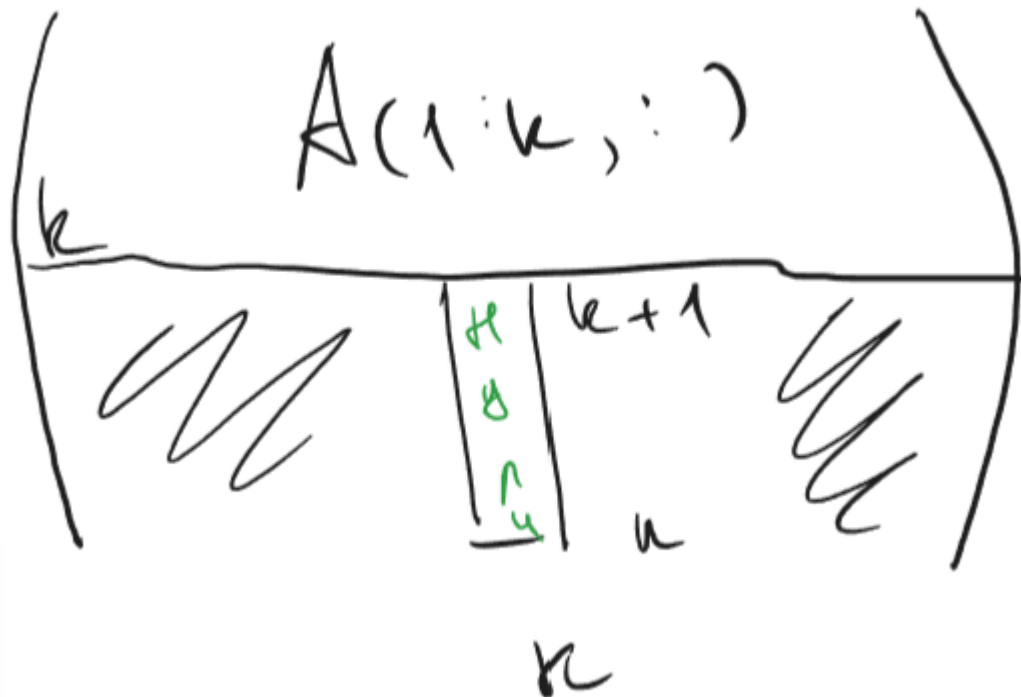
$$\tau^{(k)} l_k^T = \begin{pmatrix} 0 & \dots & 0 & \dots & 0 \\ \vdots & & \vdots & & \vdots \\ 0 & \dots & \tau_k^{(k)} & \dots & 0 \\ \vdots & & \vdots & & \vdots \\ 0 & \dots & \tau_n^{(k)} & \dots & 0 \end{pmatrix}$$

Ненулевые значения только в k -м столбце — это множители Гаусса.

$$\tau_i^{(k)} = \frac{a_{ik}}{a_{kk}}$$

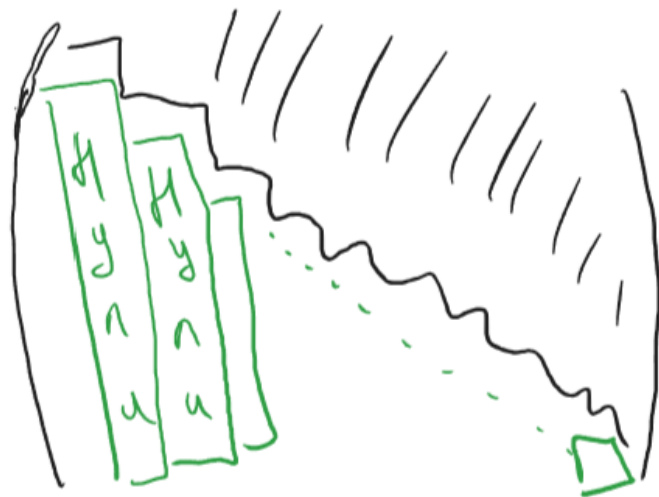
$I - \tau^{(k)} l_k^T$ — это M_k матрица преобразования Гаусса

$$M_k A = (I - \tau^{(k)} l_k^T) A = A - \tau^{(k)} l_k^T A = A - \tau^{(k)} A(k, :)$$



$$C = IAB, \text{ где } A - I \text{ на } N, B - N \text{ на } J, C - I \text{ на } J$$

$$M_{n-1} \dots M_2 M_1 A$$

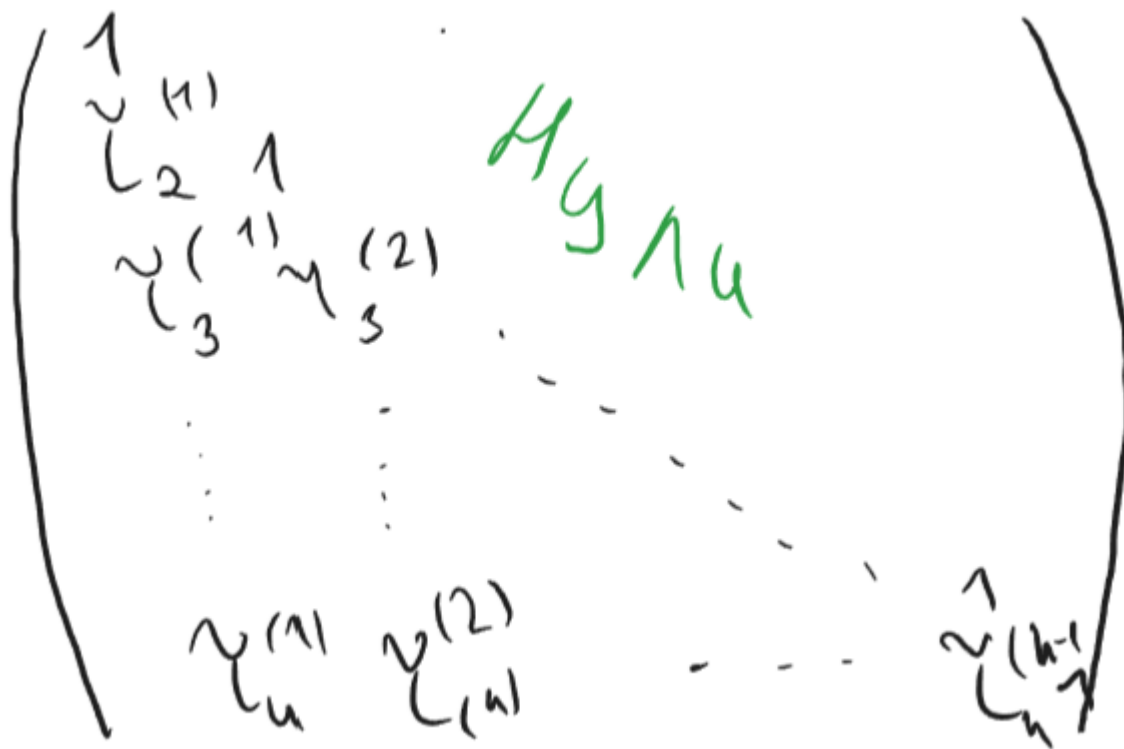


$$M_{n-1} \dots M_2 M_1 A = U$$

$$M_{n-2} \dots M_1 A = M_{n-1}^{-1} U$$

$$A = M_1^{-1} M_2^{-1} \dots M_{n-1}^{-1} U$$

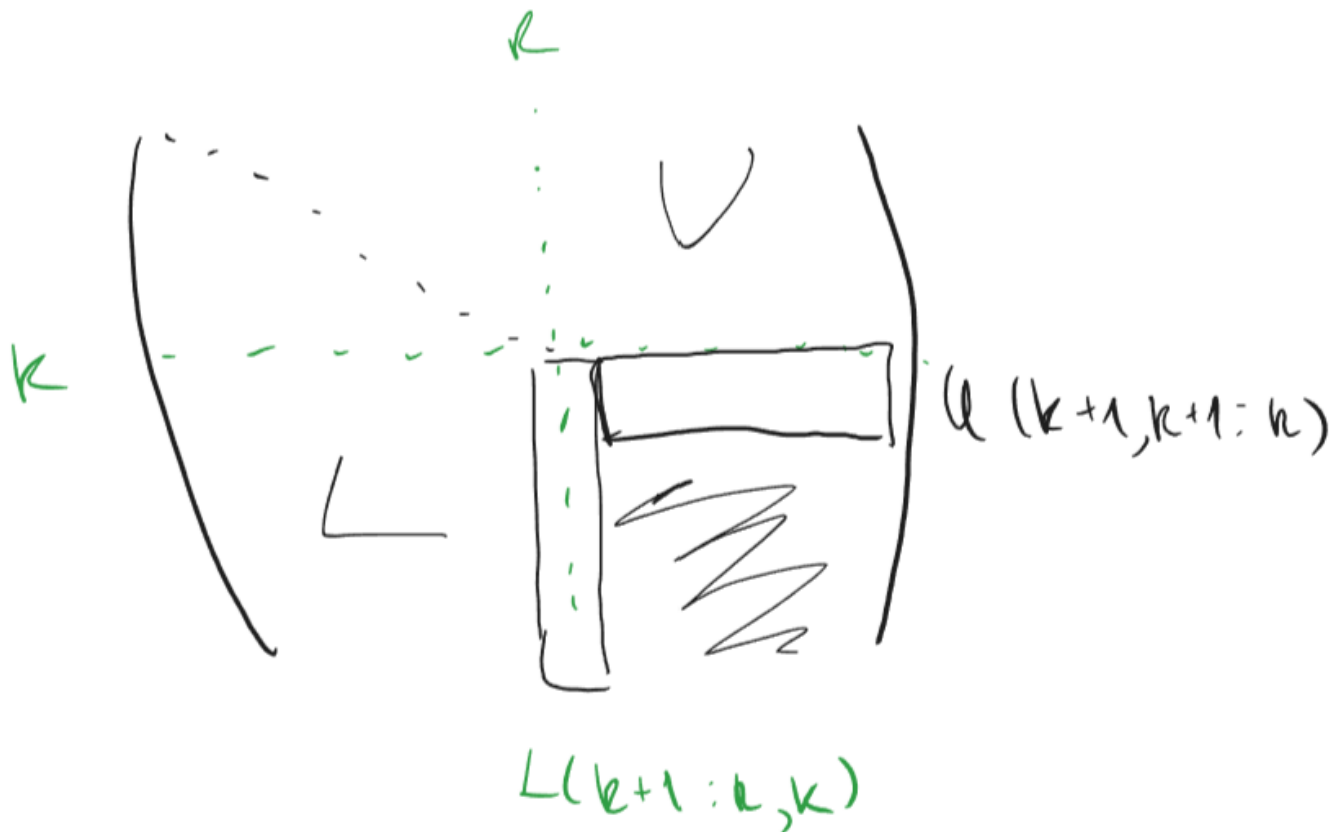
$$L = \prod_{k=1}^{n-1} M_k^{-1} = I + \sum_{k=1}^{n-1} \tau^{(k)} l_k^T$$



Алгоритм LU разложения:

```
for k 1:n-1
    A(k+1:n,k) = A(k+1:n,k) / A(k,k) - k-ый вектор множителя гаусса (k-ый столбец матрицы L)
    A(k+1:n,k+1:n) = A(k+1:n,k+1:n) - A(k+1:n,k)A(k,k+1:n)
endfor
```

На k -ом шаге алгоритма находи $k + 1$ -ый столбец:



	Название	Вектор	Матричные операции	Доступ к данным
C	kji	Столбцовый гахру	Модификация матрицы внешним произведением	Столбцовый
C	kij	Строчный гахру	Модификация матрицы внешним произведением	Строчный
Fortran	jik	Скалярное произведение	Столбцовое гахру	Смешанное
Fortran	jki	Столбцовое гахру	Столбцовое гахру	Столбцовый

При возможности реализовать одно и тоже через модификацию матрицы или гахру предпочтение отдают гахру, потому что по сравнению с модификацией матрицей гахру характеризуется вдвое меньшим объемом коммуникаций между кэш памятью процессора и регистрами

16. Векторные алгоритмы LU-разложения через гахру.

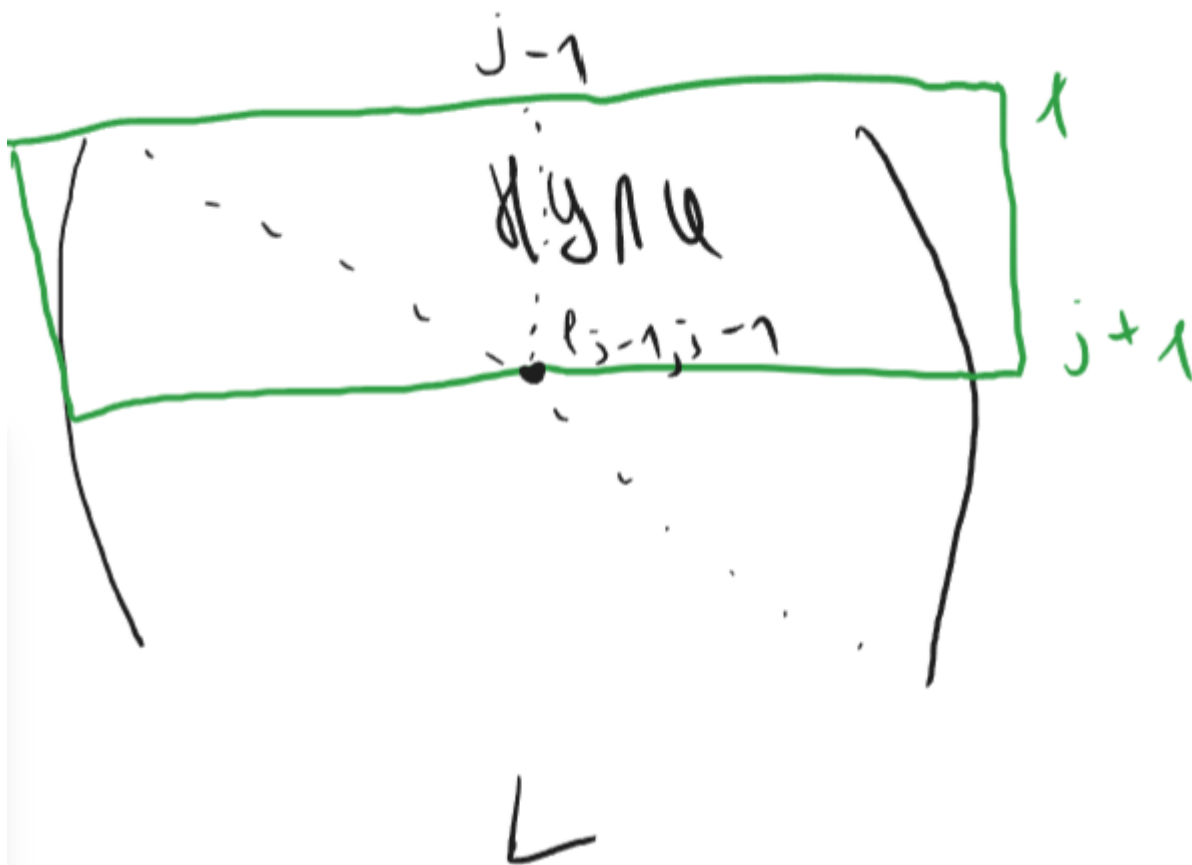
Пункт 3.2.3 Производственное LU разложение через гахру

$$A = LU$$

Положим что к j -ому шагу алгоритма первые $j - 1$ столбцов L и U уже найдены, найдем j

$$A(:, j) = LU(:, j)$$

$$1) A(1:j-1, j) = L(1:j-1, :)U(:, j)$$



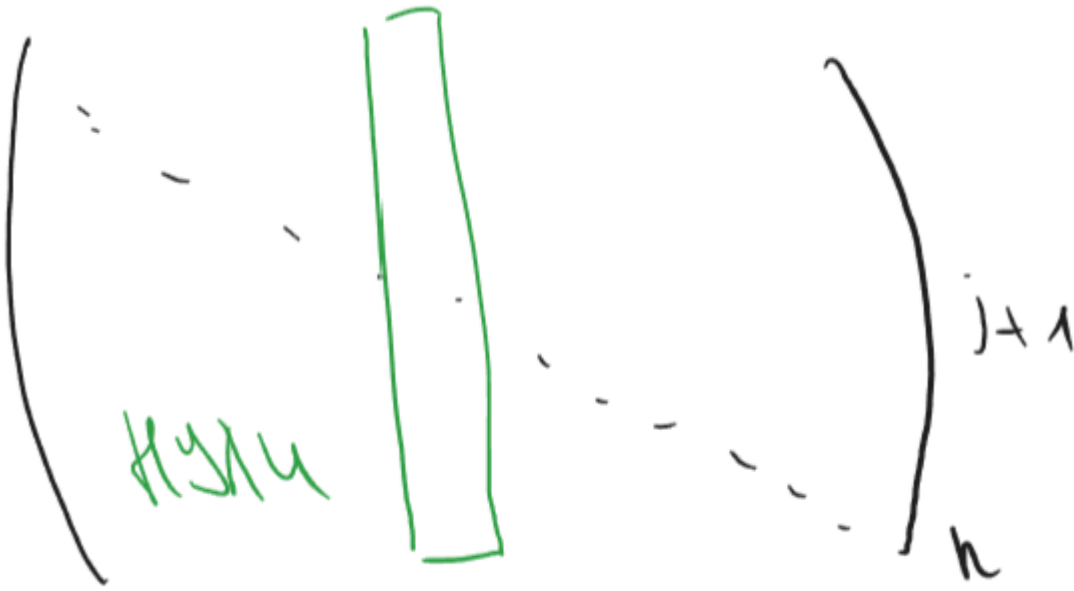
$A(1:j-1, j) = L(1:j-1, 1:j-1)U(1:j-1, j)$, получаем:

$L(1:j-1, 1:j-1)$ - знаем

$U(1:j-1, j)$ - ищем

Это СЛАУ $Lx = b$, где $b = A(1:j-1, j)$, $x = U(1:j-1, j)$

$$2) A(j:n, j) = L(j:n, :)U(:, j)$$



$$A(j : n, j) = L(j : n, 1 : j)U(1 : j, j)$$

$$A(j : n, j) = \sum_{k=1}^j L(j : n, k)U(k, j)$$

$$A(j : n, j) = \sum_{k=1}^{j-1} L(j : n, k)U(k, j) + L(j : n, j)U(j, j)$$

$$A(j : n, j) = L(j : n, 1 : j-1)U(1 : j-1, j) + L(j : n, j)U(j, j), \text{ получаем:}$$

$$A(j : n, j) - \text{знаем}$$

$$L(j : n, 1 : j-1) - \text{знаем}$$

$$U(1 : j-1, j) - \text{знаем}$$

$$L(j : n, j) - \text{неизвестно } V(j : n)$$

$$V(j : n) = A(j : n, j) - L(j : n, 1 : j-1)U(1 : j-1, j) - \text{даем}$$

$$V(j : n) = L(j : n, 1 : j-1)U(j, j)$$

$$\begin{pmatrix} \square \\ & \square \\ & & \square \end{pmatrix} = \begin{pmatrix} \square \\ & \square \\ & & 1 \end{pmatrix} \begin{pmatrix} \square \\ & \square \\ & & \square \end{pmatrix}$$

$$V(j) = L(j:j) V(j,j)$$

$$L(j+1:n, j) = V(j+1:n) U(j, j)$$

```
for j=1:n
    1) решаем СЛАУ  $L(1:j-1, 1:j-1)U(1:j-1, j) = A(1:j-1, j)$ 
    2) гаусс  $V(j:n) = A(j:n, j) - L(j:n, j-1)U(1:j-1, j)$ 
     $U(j, j) = V(j)$ 
     $L(j+1:n, j) = V(j+1:n)U(j, j)$ 
endfor
```

```
for j=1:n
    for k=1:j-2
         $A(k+1:j-1, k) = A(k+1:j-1, k) - A(k+1:j-1, k)A(k, j)$ 
    endfor
     $A(j:n, j) = A(j:n, j) - A(j:n, 1:j-1)A(1:j-1, j) \Rightarrow$  вниз смотри 1), 2)
     $A(j+1:n, j) = A(j+1:n, j)A(j, j)$ 
endfor
```

1)

```
for k=1:j-1
     $A(j:n, j) = A(j:n, j) - A(j:n, k)A(k, j)$ 
endfor
```

2)

```
for i=j:n
     $A(i, j) = A(i, j) - A(i, 1:j-1)A(1:j-1, j)$  - скалярное произведение
endfor
```

17. Векторные алгоритмы разложения Холецкого через модификацию матрицы внешним произведением.

$$A = LU$$

$$A = L \cdot L^T$$

$$A = G \cdot G^T$$

Пункт 3.3.1 Разложение Холецкого через модификацию матрицы внешним произведением

Пусть $r = 1$

$$A = \begin{pmatrix} \alpha & v^T \\ v & B \end{pmatrix} =$$
$$= \begin{pmatrix} \frac{\beta}{\alpha} & \text{нули} \\ \frac{v}{\beta} & I_{n-1} \end{pmatrix} \begin{pmatrix} 1 & \text{нули} \\ \text{нули} & B - \frac{v \cdot v^T}{\alpha} \end{pmatrix} \begin{pmatrix} \beta & \frac{v^T}{\beta} \\ \frac{v}{\beta} & I_{n-1} \end{pmatrix}$$

$$\alpha \in \mathbb{R}$$

$$\alpha = a_{11}; \beta = \sqrt{\alpha}$$

$$v = A(2:n, 1)$$

$$v \in \mathbb{R}^{(n-1) \times 1}$$

$$B = A(2:n, 2:n)$$

$$B \in \mathbb{R}^{(n-1) \times (n-1)}$$

$$B - \frac{v \cdot v^T}{\alpha} - \text{модификация матрицы внешним произведением}$$

$$B - \frac{v \cdot v^T}{\alpha} = G_1 \cdot G_1^T = \begin{pmatrix} \frac{\beta}{\alpha} & \text{нули} \\ \frac{v}{\beta} & G_1 \end{pmatrix} \begin{pmatrix} \beta & \frac{v^T}{\beta} \\ \text{нули} & G_1^T \end{pmatrix}$$

$$G_1 \in \mathbb{R}^{(n-1) \times (n-1)}$$

$$G = \begin{pmatrix} \frac{\beta}{\alpha} & \text{нули} \\ \frac{v}{\beta} & G_1 \end{pmatrix}$$

Матричный вид:

```
for k=1:n
    A(k:n,k) = A(k:n,k)(sqrt(A(k,k)))
    A(k+1:n,k+1:n) = A(k+1:n,k+1:n) - A(k+1:n,k)[A(k+1:n,k)]^T
endfor
```

Векторный вид:

```
for k=1:n
    A(k:n,k) = A(k:n,k)/sqrt(A(k,k))
    for j=k+1:n
        A(j:n,j) = A(j:n,j) - A(j:n,k)A(j,k)
    endfor
endfor
```

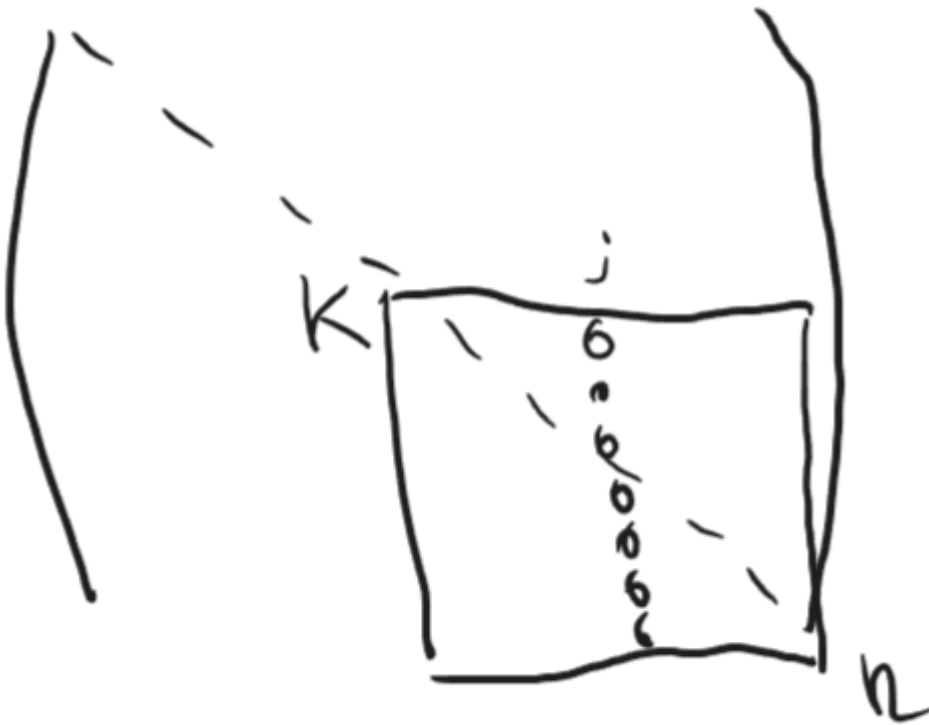


Таблица для Холецкого:

Название алгоритма	Векторная операция	Матричная операция	Доступ к памяти
kji	столбцовый гахру	модификацию матрицы внешним произведением	по столбцам
kij	строчный гахру	модификацию матрицы внешним произведением	по строкам

18. Векторные алгоритмы разложения Холецкого через гахру.

Пункт 3.3.2 Разложение Холецкого через гахру

Пусть к j -ому шагу алгоритма первые $j - 1$ -ый столбцов матрицы J найдены, отыщем j -ий.

$$I \begin{pmatrix} \overset{j}{c} \end{pmatrix} = \begin{pmatrix} I & A \end{pmatrix} \begin{pmatrix} \overset{j}{a} \\ B \end{pmatrix}$$

$$A = G * G^T \quad A(:, j) = G * G^T(:, j)$$

$$1) A(1 : j - 1, j) = G(1 : j - 1, :) G^T(:, j)$$

$$A(1:j-1, j) = G(1:j-1, 1:j-1)G^T(1:j-1, j)$$

$$2) A(j:n, j) = G(j:n, j)G^T(:, j)$$

$$A(j:n, j) = G(j:n, 1:j)G^T(1:j, j)$$

$$A(j:n, j) = \sum_{k=1}^J G(j:n, k)G^T(k, j)$$

$$A(j:n, j) = \sum_{k=1}^{j-1} G(j:n, k)G^T(k, j) + G(j:n, j)G^T(j, :)$$

$$A(j:n, j) = G(j:n, 1:j-1)G^T(1:j-1, j) + G(j:n, j)G^T(j, j)$$

$$G(j:n, j)G^T(j, j) \quad (\text{не знаем, назовём} - V(j:n))$$

$$V(j:n) = A(j:n, j) - G(j:n, 1:j-1)G^T(1:j-1, j)$$

$$V(j:n) = G(j:n, j)G^T(j, j)$$

$$V(j:n) = G(j, j)G(j, j)$$

$$G(j, j) = \sqrt{V(j)}$$

$$G(j+1:n, j) = V(j+1:n)G(j, j)$$

Алгоритм:

```
for j=1:n
    A(j:n, j) = A(j:n, j) - A(j:n, 1:j-1) * [A(j, 1:j-1)]^T #столбцовый гахру
    #V(j, n)    #A(j:n, j)    #G(j:n, 1:j-1)    #G^T(1:j-1, j)

    A(j:n, j) = A(j:n, j) / sqrt(A(j, j))
endfor
```

преобразуем операцию гахру через скалярное произведение (Алгоритм jik) $\Rightarrow$:

```
for i=j:n
    A(i, j) = A(i, j) - A(i, 1:j-1) * [A(j, 1:j-1)]^T
endfor
```

преобразуем операцию гахру через столбцовое сахру (Алгоритм jki) $\Rightarrow$:

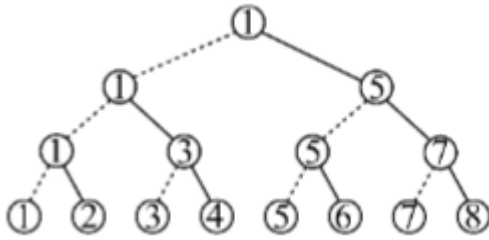
```
for k=1:j-1
    A(j:n, j) = A(j:n, j) - A(j:n, k) A(j, k)
endfor
```

	Название алгоритма	Векторная операция	Матричная операция	Доступ к памяти
C	jik	Скалярное произведение	Столбцовое гахру	Строчный
C/Fortran	jki	Столбцовое сахру	Столбцовое гахру	Столбцовый

ПАРАЛЛЕЛЬНЫЕ АЛГОРИТМЫ

19. Рассылка данных по бинарному дереву.

Бинарное дерево



Звезда



В сетевой конфигурации звезда выделяется центральный процессор (manager), связанный с каждым из периферийных (workers). Её недостатком признается необходимость маршрутизации через единственный узел.

Поэтому в задачах, связанных с рассылкой или сбором данных, уместней использование конфигурации бинарное дерево, на которой принято реализовывать следующую стратегию «разделяй и властвуй».

Положим, что необходимо разослать некоторое сообщение от одного процессора остальным; всего процессоров 8. Тогда на конфигурации звезда и бинарное дерево это займет по 7 коммуникаций, однако в последнем случае часть коммуникаций можно произвести одновременно.

Действительно, на первом шаге процессор 1 отправляет сообщение процессору 5, на втором шаге одновременно 1 отправляет 3, а 5 – 7. Завершают рассылку коммуникации между нечетными (отправляют) и четными (принимают) процессорами. Последовательно производится лишь 3 коммуникации. В общем случае вместо $p - 1$ коммуникации на звезде получаем $\log_2 p$ на бинарном дереве, где p – число процессоров.

Рассматривая пример бинарного дерева на рисунке для $p=8$, выделим на нем четыре уровня, нумеруя их сверху вниз 3, 2, 1, 0 в порядке убывания.

На третьем (верхнем) расположена задача 1, которой предстоит начать вычисления, отправив задаче 5 правую половину вектора $a(n/2+1:n)$.

На втором уровне задача 1 отправляет задаче 3 вторую четверть вектора $a(n/4+1:n/2)$, задача 5 принимает вторую половину a от задачи 1 и отправляет последнюю четверть $a(3/4 \cdot n+1:n)$ задаче 7.

Действия на первом уровне характеризуются отправкой задачей 1 второй восьмушки вектора $a(n/8+1:n/4)$ задаче 2; задачи 3 и 7 принимают от 1 и 5 вектора $a(n/4+1:n/2)$ и $a(3/4 \cdot n+1:n)$ соответственно; затем задачи 3, 5 и 7 отправляют задачам 4, 6 и 8 вектора $a(3/8 \cdot n+1:n/2)$, $a(5/8 \cdot n+1:3/4 \cdot n)$ и $a(7/8 \cdot n+1:n)$.

На нижнем уровне задачи с четными номерами принимают отправленные им ранее данные.

Теперь необходимо сформулировать это в виде алгоритма, не ограничиваясь случаем восьми задач. Осмыслив проблему в целом, разделим задачи искомого алгоритма на три множества: первая задача (несколько раз отправляет данные), четные задачи (один раз принимают) и все остальные (один раз принимают, несколько раз отправляют). Сформулировав инструкции для первой и четных задач можно объединить их и предписать получившиеся действия остальным задачам. В итоге получится Алгоритм 1.2.

Первой задаче предписывается исполнение инструкций в строках 02-06, где она в цикле по i , спускаясь по уровням дерева, рассылает вторую половину оставшегося у нее вектора a (вектор на каждой итерации сокращается вдвое с отсечением конца) задаче $1 + 2^{i-1}$. Все четные задачи исполняют инструкцию 09, принимая известное количество элементов вектора у задачи с номером на единицу меньше.

```

# инициализация
a(1:n) – рассылаемый вектор,
n – его размер,
μ – номер задачи,
p – общее количество задач,
r = n/p, топология ком- муникаций – бинарное дерево
01 if μ=1 then # действия первой задачи
    02 k=log2p; # верхний уровень дерева
    03 for i=k:-1:1 # организация коммуникаций
        04 send(a(n/2+1:n), 1+2^{i-1} ); # отправка данных
        05 n=n/2; # сокращение длины вектора
    06 endfor
07 else
    08 if μ - четное then # действия четных задач
        09 recv(a(1:r), μ-1); # прием данных
    10 else # действия оставшихся задач
        11 # поиск уровня k, с которого начинается рассылка
        12 k=0; q=μ;
        13 while mod(q,2)≠0
            14 q=(q+1)/2; k=k+1;
        15 endwhile
        16 n=n/2^{log2p-k}; # размер принимаемых данных
        17 recv(a(1:n), μ-2k); # прием данных
        18 for i=k:-1:1 # организация коммуникаций
            19 send(a(n/2+1:n), μ+2^{i-1} ); # отправка данных
            20 n=n/2; # сокращение длины вектора
        21 endfor
    22 endif
23 endif

```

Нечетные, за исключением первой, сначала в 11–15 по знакомой из теории графов процедуре вычисляют уровень дерева k , на котором необходимо произвести прием. Затем определяют в зависимости от этого уровня количество принимаемых данных (строка 16) и производят прием (17) от задачи $\mu - 2^k$ – фрагмент, аналогичный действиям четных задач. Завершается алгоритм рассылкой этих данных по дереву (18–21), как в первой задаче.

20. Параллельная реализация рекуррентных выражений.

Циклические конструкции широко применяются для организации вычислений по рекуррентным выражениям.

Рассмотрим на примере рекуррентного выражения $x_i = a_i \cdot x_{i-1}$ ($1 \leq i \leq n$) при необходимости отыскать x_n . На пространстве итераций $I = \{(i) : 1 \leq i \leq n\}$ каждый вектор зависит от всех предыдущих (пространство упорядочено отношением следования) и нет никаких двух независимых друг от друга векторов. Методы автоматического распараллеливания не подходят для данной задачи. И все же такой алгоритм очевиден.

Представим рассматриваемое рекуррентное выражение в «развернутом» виде

$$x_n = a_n \cdot a_{n-1} \cdot a_{n-2} \cdot \dots \cdot a_3 \cdot a_2 \cdot a_1 \cdot x_0$$

и поставим в соответствие задаче μ ($1 \leq \mu \leq n/2$) параллельного алгоритма операцию $a_{2\mu} \cdot a_{2\mu-1}$. Все $n/2$ таких операций на первом шаге параллельного алгоритма могут быть произведены одновременно — параллельно. Далее возможно одновременное исполнение $n/4, n/8, \dots, 2$ умножений — всего за $\log_2(n) - 1$ шагов параллельного алгоритма. На завершающем шаге результат домножается на x_0 .

Ускорение вычислений по такому алгоритму оцениваем величиной $n / \log_2(n)$.

Приведенный пример с одной стороны иллюстрирует превосходство живого человеческого воображения над шаблонным подходом, с другой стороны указывает на необходимость формализации процедуры распараллеливания вычислений по рекуррентным выражениям.

Будем двигаться в сторону усложнения рекуррентного выражения.

Рассмотрим случай $x_i = a_i \cdot x_{i-1} + b_i$ ($1 \leq i \leq n$) **при необходимости отыскать** x_n . Применить для его параллельного вычисления прием, аналогичный предыдущему, удастся после представления рассматриваемого выражения в форме без последнего слагаемого. Для этого перейдем к матричной нотации.

Пусть вектор $\mathbf{x}_i = \begin{pmatrix} x_i \\ 1 \end{pmatrix}$ и матрица $A_i = \begin{pmatrix} a_i & b_i \\ 0 & 1 \end{pmatrix}$. Тогда рекуррентное выражение запишется в виде $\mathbf{x}_i = A_i \mathbf{x}_{i-1}$. И $\mathbf{x}_n = A_n \cdot A_{n-1} \cdot A_{n-2} \cdot \dots \cdot A_3 \cdot A_2 \cdot A_1 \cdot \mathbf{x}_0$ после «развертывания». Умножение матриц можно провести параллельно по ранее представленной процедуре.

Структура матрицы, характеризующейся второй строчкой (0 1), при умножении воспроизводится, следовательно лишних арифметических операций не будет.

При работе с выражением $x_i = a_i \cdot x_{i-1} + b_i \cdot x_{i-2}$ уместно представить вектор $\mathbf{x}_i = \begin{pmatrix} x_i \\ x_{i-1} \end{pmatrix}$ и матрицу $A_i = \begin{pmatrix} a_i & b_i \\ 1 & 0 \end{pmatrix}$. Тогда рекуррентное выражение по-прежнему запишется в виде $\mathbf{x}_i = A_i \mathbf{x}_{i-1}$.

Далее, для рекуррентного выражения $x_i = a_i \cdot x_{i-1} + b_i \cdot x_{i-2} + c_i$ размерность вектора и матрицы повысится:

$$\mathbf{x}_i = \begin{pmatrix} x_i \\ x_{i-1} \\ 1 \end{pmatrix}, \quad A_i = \begin{pmatrix} a_i & b_i & c_i \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

В общем случае, для выражения $x_i = a_i \cdot x_{i-1} + b_i \cdot x_{i-2} + \dots + c_i \cdot x_{i-k+1} + d_i \cdot x_{i-k} + e_i$ записывают:

$$\mathbf{x}_i = \begin{pmatrix} x_i \\ x_{i-1} \\ \dots \\ x_{i-k+1} \\ x_{i-k} \\ 1 \end{pmatrix}, \quad A_i = \begin{pmatrix} a_i & b_i & \dots & c_i & d_i & e_i \\ 1 & 0 & \dots & 0 & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & \dots & 0 & 0 & 1 \end{pmatrix}$$

21. !Систолический алгоритм гахру на процессорном кольце с декомпозицией матрицы на блочные строки.!

Систолические алгоритмы характеризуются следующей структурой:

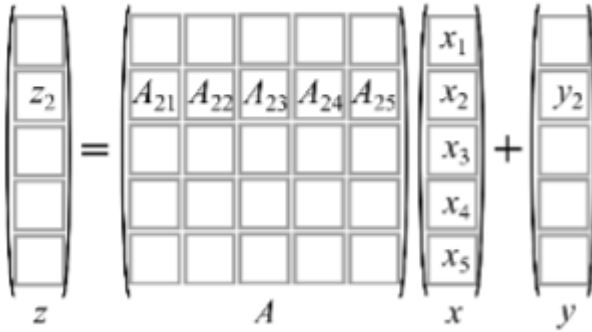
```
for t=1:T #глобальный
    send(...);
    recv(...);
    арифметические операции
end
```

Операцию гахру (general A x plus y.) принято записывать в виде:

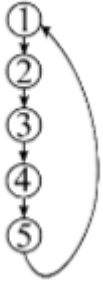
$$\begin{aligned} &\in R^{n \times n} \\ x, y, z &\in R^{n \times 1} \\ 1 \leq \mu &\leq p \\ z &= Ax + y \end{aligned}$$

Положим, что вектора и матрица из гахру для удобства составления параллельного алгоритма разбиты на блоки:

$$\begin{aligned} x_i, y_i, z_i &\in R^{\alpha \times 1} \\ 1 \leq i &\leq p \\ \alpha &= \frac{n}{p} \end{aligned}$$



задачи



Тогда выделяя из строки x отдельные блоки перепишем последнее выражение через блочное скалярное произведение блочной строки A и вектора x :

$$z_{\mu} = A_{\mu}x + y_{\mu} = \sum_{k=1}^p A_{\mu k}x_k + y_{\mu}$$

Необходимости в обмене данными между задачами при таком их распределении не возникает; платой за отсутствие коммуникаций является дублирование вектора x в памяти всех задач.

Алгоритм 1 Параллельный алгоритм гахру без коммуникаций с распределением матрицы по блочным строкам:

```
# инициализация
row = (mu-1)*alpha+1:mu*alpha
A_loc = A(row,:)
y_loc = y(row)
x_loc = x
for k = 1:p
    a = (k-1)*alpha+1:k*alpha
    y_loc = y_loc + A_loc(:,a)*x_loc(a)
endfor
```

Переходя к построению следующего алгоритма откажемся от дублирования вектора x в памяти всех задач. Пусть при начальном распределении данных задаче μ достанется блок x_{μ} . Тогда необходимо предусмотреть обмен блоками этого вектора между задачами алгоритма; такой, чтобы в памяти любой задачи побывали все блоки x , как необходимые для вычислений.

Алгоритм 2 Параллельный алгоритм гахру на кольце с распределением матрицы по блочным строкам:

```

# инициализация:
row = (mu-1)*alpha+1:mu*alpha
A_loc = A(row,:)
y_loc = y(row)
x_loc = x(row)
left, right
for k = 1:p
    send(x_loc, right)
    recv(x_loc, left)
    tau = mu - k;
    if tau ≤ 0 then
        tau=tau+p
        a = (tau-1)*alpha+1:tau*alpha
    endif
    y_loc = y_loc+A_loc(:,a)*x_loc
endfor

```

!!!Перефразированный текст из методички нейронкой!!!

В новом варианте инициализации каждая задача μ хранит только один блок вектора x :

$$x_{\mu} = x((\mu - 1)\beta + 1 : \mu\beta).$$

Переменные $left$ и $right$ задают соседние задачи в процессорном кольце:

$$left = \mu - 1, \quad right = \mu + 1,$$

при этом для первой задачи левым соседом является задача p , а для последней правым — задача 1.

На каждой итерации задача отправляет текущий блок x_{loc} правому соседу ($send$) и получает новый блок от левого соседа ($recv$). За p итераций через каждую задачу проходят все блоки вектора x .

Индекс полученного блока определяется как τ . Если $\tau \leq 0$, то:

$$\tau = \tau + p,$$

что учитывает циклический переход между задачами p и 1.

В зависимости от τ выбирается соответствующий блок матрицы:

$$A_{\mu\tau},$$

и вычисляется часть суммы:

$$y_{\mu} = \sum_{\tau=1}^p A_{\mu\tau} x_{\tau}.$$

В отличие от Алгоритма 1, где все задачи одновременно использовали один и тот же блок x , в Алгоритме 2 на каждой итерации используются разные блоки. Например, на первой итерации выполняются:

$$A_{1p}x_p, \quad A_{21}x_1, \quad \dots, \quad A_{p,p-1}x_{p-1},$$

а на последней:

$$A_{11}x_1, \quad A_{22}x_2, \quad \dots, \quad A_{pp}x_p.$$

Таким образом, вычисления организуются как циклический обмен блоками вектора x между задачами без их дублирования в памяти.

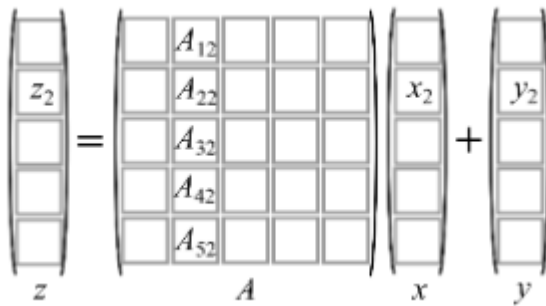
22. !Систолический алгоритм дахру на процессорном кольце с декомпозицией матрицы на блочные строкицы.!

Систолические алгоритмы характеризуются следующей структурой:

```

for t=1:T #глобальный
    send(...);
    recv(...);
    арифметические операции
end

```



Линейная комбинация блочных столбцов матрицы A :

$$z = \sum_{j=1}^p A_j x_j + y,$$

где блочный столбец

$$A_j \in \mathbb{R}^{n \times \beta}, \quad 1 \leq j \leq p.$$

Необходимо распределить вычисления данной операции между p задачами параллельного алгоритма с учетом коммуникаций.

Предполагается, что каждая задача μ вычисляет блок

$$z_\mu.$$

Тогда при

$$\omega_j = A_j x_j$$

выражение для z_μ принимает вид:

$$z_\mu = \sum_{j=1}^p \omega_j(\text{row}) + y_\mu.$$

Здесь

$$\omega_j \in \mathbb{R}^{n \times 1},$$

$$\text{row} = (\mu - 1)\alpha + 1 : \mu\alpha.$$

Алгоритм 3:

```

# инициализация
col=(μ-1)α+1:μα;
Aloc=A(:,col);
xloc=x(col);
yloc=y(col);
left, right

w=Aloc xloc

for j=1:p
    send(w,right)
    recv(w,left)
    yloc=yloc+w(col)
end for

```

!!!Перефразированный текст из методички нейронкой!!!

После инициализации каждая задача μ содержит блочный столбец A_μ и соответствующий блок вектора x_μ . Поэтому сначала выполняется локальное умножение:

$$\omega_\mu = A_\mu x_\mu.$$

Полученный вектор

$$\omega_\mu$$

затем передается по процессорному кольцу. За время выполнения цикла через каждую задачу проходят все векторы

$$\omega_j$$

из выражения

$$z_\mu = \sum_{j=1}^p \omega_j(row) + y_\mu.$$

На каждой итерации, помимо коммуникации, выполняется одно сложение:

$$y_{loc} = y_{loc} + \omega(row).$$

Как и в Алгоритме 2, порядок сложения в разных задачах различается, однако это не влияет на результат благодаря коммутативности сложения векторов.

В отличие от Алгоритма 2, здесь не требуется учитывать номер задачи, сформировавшей принятый вектор ω . Все полученные векторы обрабатываются одинаково, что существенно упрощает алгоритм.

Недостатком такого подхода является увеличение объема передаваемых данных. Вместо блоков

$$x_{loc} \in \mathbb{R}^\beta$$

по кольцу пересылаются полные векторы

$$\omega \in \mathbb{R}^n,$$

из-за чего объем коммуникаций возрастает в

$$\frac{n}{\beta}$$

раз.

23. !Столбцово-ориентированные алгоритмы умножения матриц на кольце.!

Далее будем говорить о производстве операции

$$D = C + AB,$$

где

$$A, B, C, D \in R^{\alpha \times \alpha}$$

$$A_{ij}, B_{ij}, D_{ij}, C_{ij} \in R^{n \times n}$$

$$1 \leq i, j \leq p, \quad \alpha = n/p,$$

Задача μ :

$$D_\mu = C_\mu + AB_\mu = C_\mu + \sum_{j=1}^p A_j B_{j\mu}.$$

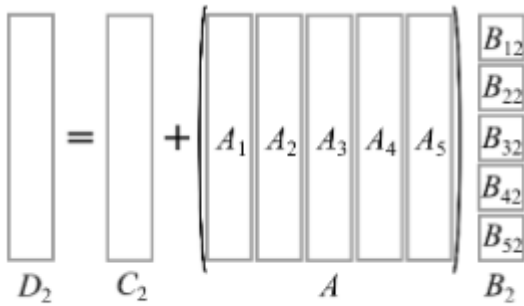
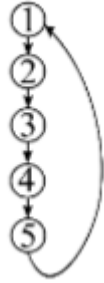


Рис. 4. Иллюстрация к ф. (4) при $\mu=2$ и $p=5$

задачи



В Алгоритме матрица A распределяется по блочным столбцам, а вычисления по формуле выше сопровождаются пересылками упомянутых столбцов между задачами. Для организации таких пересылок используются каналы, соответствующие топологии процессорное кольцо.

```
# Инициализация
col = (mu - 1)*alpha + 1:mu*alpha;
Aloc = A(:, col);
Bloc = B(:, col);
Cloc = C(:, col);
left, right

for j = 1:p
    send(Aloc, right)
    recv(Aloc, left)
    # определение индекса полученного блока A_tau
    tau = mu - j;
    if tau <= 0 then
        tau = tau + p;
    end if
    a = (tau - 1)*alpha + 1:tau*alpha;
    # индексы, задающие блочный столбец tau матр. A
    Cloc = Cloc + Aloc * Bloc(a, :);
endfor
```

24. !Строчно-ориентированные алгоритмы умножения матриц на кольце.!

При умножении прямоугольных матриц A и B может случиться, что элементов в A значительно больше, чем в B (при $n \gg m$). Тогда для сокращения коммуникационных издержек разумнее пересылать по кольцу блоки матрицы B , а не A

$$ID^J = IA^N \cdot NB^J + IC^J$$

$I \gg J$ используем строчно-ориентированный алгоритм $J \gg I$ используем алгоритм столбцовый, из вопросы выше

Далее будем говорить о производстве операции

$$D = C + AB,$$

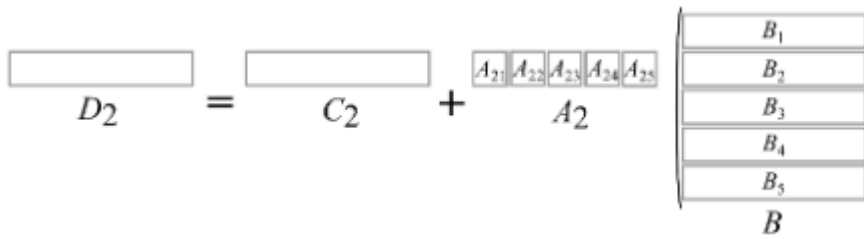
где

$$A, B, C, D \in R^{\alpha \times \alpha}$$

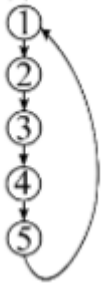
$$A_{ij}, B_{ij}, D_{ij}, C_{ij} \in R^{n \times n}$$

$$1 \leq i, j \leq p, \quad \alpha = n/p,$$

$$D_\mu = C_\mu + AB_\mu = C_\mu + \sum_{i=1}^p A_i B_i,$$



задачи



```
# инициализация
row = (mu - 1)alpha + 1 : mu*alpha
Aloc = A(row,:)
Bloc = B(row,:)
Cloc = C(row,:)
left, right

for j = 1 : p
    send(Bloc, right)
    recv(Bloc, left)
    # определение индекса полученного блока B_tau
    tau = mu - j;
    if tau <= 0 then
        tau = tau + p
    endif
    a = (tau - 1)alpha + 1:tau*alpha;
    # индексы, задающие блочную строку tau матр. B
    Cloc = Cloc + Aloc(:, a) * Bloc;
end for
```

25. !Систолический алгоритм умножения матриц на торе!

Далее будем говорить о произведении операции

$$D = C + AB,$$

На основе линейной комбинации блочных столбцов и строк:

$$D_{\mu,\lambda} = C_{\mu,\lambda} + A_{\mu}B_{\lambda} = C_{\mu,\lambda} + \sum_{k=1}^N A_{\mu,k}B_{k,\lambda}$$

где пара (μ, λ) — двузначный номер задачи ($1 \leq \mu \leq N$ и $1 \leq \lambda \leq N$, $N = \sqrt{p}$), блоки с двойными индексами $A_{\mu,k}, B_{\mu,\lambda}, D_{\mu,\lambda}, C_{\mu,\lambda} \in \mathbb{R}^{\alpha \times \alpha}$, ($\alpha = n/N$);

Стремясь исключить дублирование данных и снизить объем занимаемой памяти разработаем алгоритм с коммуникациями на торе.

Пусть отправка блоков матрицы A производится в горизонтальном направлении с востока на запад, блоков матрицы B в вертикальном — с юга на север. Задачи, расположенные на крайнем западе тора связаны каналами с задачами крайнего востока, расположенные на крайнем севере связаны с задачами крайнего юга.

Тогда у задачи (μ, λ) будет четыре соседа: западный — $(\mu, \lambda - 1)$, при $\lambda = 1$ (μ, N) ; северный — $(\mu - 1, \lambda)$, при $\mu = 1$ (N, λ) ; восточный — $(\mu, \lambda + 1)$, при $\lambda = N$ $(\mu, 1)$ и южный — $(\mu + 1, \lambda)$, при $\mu = N$ $(1, \lambda)$.

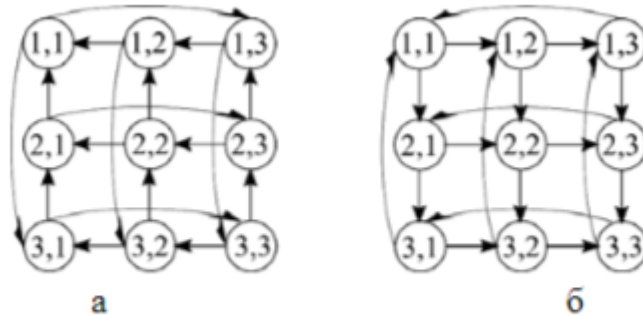


Рис. 7. Топология коммуникаций параллельных алгоритмов умножения матриц на торе: а – с отправкой данных на запад и север; б – с отправкой на восток и юг

Важной особенностью искомого алгоритма является начальное распределение данных. Необходимо, чтобы перед началом вычислений в памяти задачи (μ, λ) находились блоки $C_{\mu,\lambda}$, $A_{\mu,\rho}$ и $B_{\rho,\lambda}$, где $\rho = \langle \lambda + \mu - 1 \rangle_N$. Последняя операция связана с делением с остатком значения $\lambda + \mu - 1$ (делимое) на N (делитель). При нулевом остатке положим $\rho = N$, при отличном от нуля примем ρ равным этому остатку.

Если предварительно отнести к задаче (μ, λ) блоки $A_{\mu,\lambda}$ и $B_{\mu,\lambda}$, то в ходе начального распределения данных необходимо сдвинуть циклически μ -ую блочную строку матрицы A на $\mu - 1$ позицию влево, а блочный столбец λ матрицы B на $\lambda - 1$ позицию вверх.

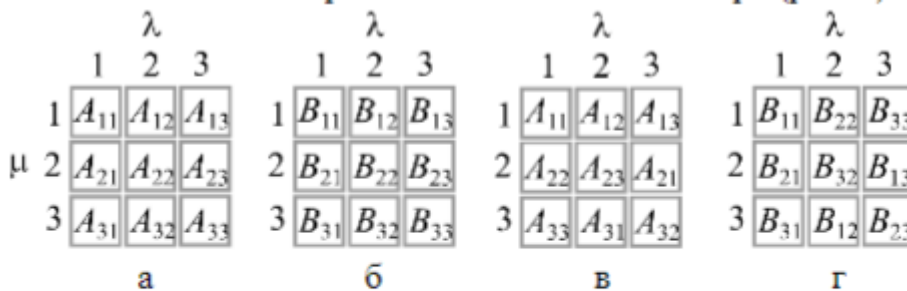


Рис. 8. Распределение блоков умножаемых матриц между задачами:

а – «естественное», но неправильное распределение матрицы A ; б – «естественное», но неправильное распределение матрицы B ; в – правильное распределение матрицы A ; г – правильное распределение матрицы B

Алгоритм с неправильным распределением:


```

# инициализация
row=(mu-1)*alpha+1:mu*alpha;
col=(lambda-1)*beta+1:lambda*beta;

Aloc=A(row,col);
Bloc=B(row,col);

east, west, north, south

for k=1:mu-1 # циклическое смещение блочной строки  $\mu$  матрицы A
    send(Aloc, west); recv(Aloc, east) # на  $\mu-1$  позицию влево
endfor

for k=1:lambda-1 # циклическое смещение блочн. столбца  $\lambda$  матрицы B
    send(Bloc, north); recv(Bloc, south) # на  $\lambda-1$  позицию вверх
endfor

```

Алгоритм с правильным распределением:

```

# инициализация
row=( $\mu$ -1)*alpha+1: $\mu$ *alpha;
a=(beta-1)*alpha+1:beta*alpha
col=(lambda-1)*alpha+1:lambda*alpha;

Cloc=C(row,col);
Aloc=A(row,a);
Bloc=B(a,col);

east, west, north, south;

for k=1:N
    send(Aloc, west)
    send(Bloc, north)
    recv(Aloc, east)
    recv(Bloc, south)
    Cloc=Cloc+Aloc*Bloc
endfor

```

26. Параллельный алгоритм разложения Холецкого на кольце.

Далее будем говорить о выводе параллельного алгоритма разложения Холецкого $A = GG^T$, где матрица $A \in \mathbb{R}^{n \times n}$ — симметричная положительно определенная, а $G \in \mathbb{R}^{n \times n}$ — нижняя треугольная с положительными элементами на главной диагонали.

Алгоритм:

```

# инициализация
col = mu:p:n
L=length(col)
A_loc=A(:,1:L)
left, right
j = 1
l = 1
# локальный индекс столбца J подлежит вычислению в первую очередь любой задачей
# индекс столбца J только вычисляется или подлежит вычислению только
# в первую очередь той задачей в локальной памяти которой он хранится
# j у всех одинаковый, l может быть у всех разный
while l ≤ L then
    # произвольная задача не выйдет из цикла до тех пор
    # пока не вычислит все свои локальные столбцы,
    # в одной итерации цикла мы вычисляем j-й столбец и используем его максимальным образом

    if j = col(l) then
        A_loc(j:n,l)=A_loc(j:n,l)*sqrt(A_lk(j,l))
        if j≠n then
            send(A_lk(j:n,l),right)
        endif

        for g=l+1:L
            r=col(g)
            A_loc(r:n,q)=A_loc(r:n,q)-A_loc(r:n,l)*A_loc(r,l)
        endfor
        j=j+1
        l=l+1
    else
        recv(g(j:n),left)
        if (right≠alpha) and (j<beta) then
            # alpha - номер задачи, породившая j-й столбец
            # alpha можно не вычислять, а передавать по кольцу вместе с индексом j
            # beta - глобальный индекс последнего локального столбца правого соседа

            send(g(j:n),right)
        endif
        for q=l:L
            r=col(q)
            A_loc(r:n,q)=A_loc(r:n,q)-g(r:n)*g(r)
        endfor
        j=j+1
        l=l
    endif
endwhile

```

27. ???Параллельный алгоритм LU-разложения на кольце.

28. ???Параллельный алгоритм метода прогонки.

ХАРАКТЕРИСТИКИ ПАРАЛЛЕЛЬНЫХ АЛГОРИТМОВ

29. Ускорение, эффективность, масштабируемость, закон Амдала.

Ускорение (speedup): $S = \frac{T'}{T^p}$

T' - время работы последовательного

T^p - время работы параллельного, p - число задач

Смысл ускорения - во сколько раз вычисления по параллельному алгоритму производятся быстрее чем по последовательному

Эффективность (effecintly): $E = \frac{S}{p} = \frac{T'}{T^p p} * 100\%$

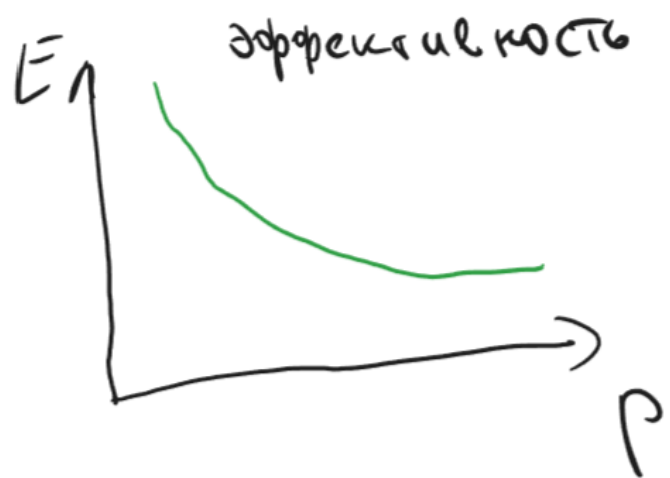
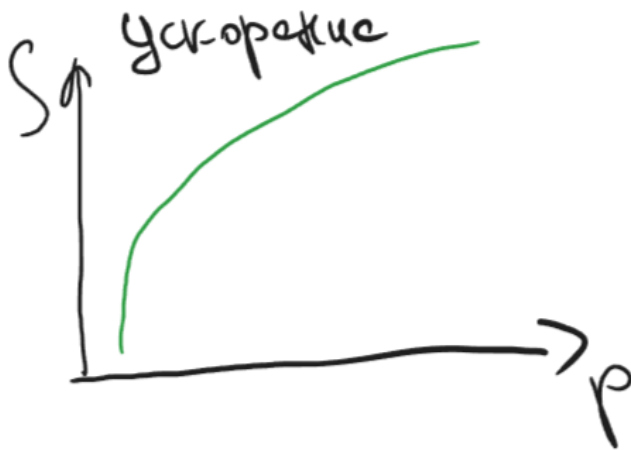
Смысл эффективности - указать какая доля исполнительных операций занимается полезными арифметическими операциями

Масштабируемость (scaleability): $M = \frac{V^p}{V'}$, при $T' = T^p$

V^p -объем памяти

V' - объем памяти при последовательном алгоритме

Смысл масштабируемости - это свойство алгоритма допускающее эффективное задействование большего количества исполнительных устройств с ростом объемов входных данных



Пункт 4.1.2.1 Закон Амдала (1957)

t' - время выполнения операций, которые не распараллелить

$\gamma = \frac{t'}{T'}$ - доля не распараллеленных операций в общем количестве

$$T^p = \gamma T' + \frac{1-\gamma}{p} T'$$

Тогда ускорение:

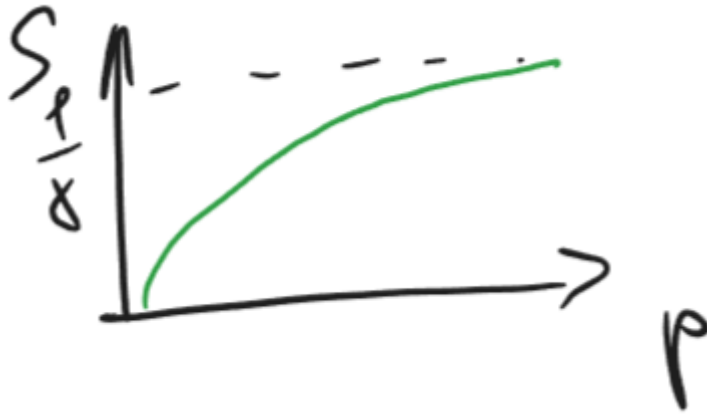
$$S = \frac{T'}{\gamma T' + \frac{1-\gamma}{p} T'} = \frac{1}{\gamma + \frac{1-\gamma}{p}}$$

Закон Амдала: $S \leq \frac{1}{\gamma + \frac{1-\gamma}{p}}$

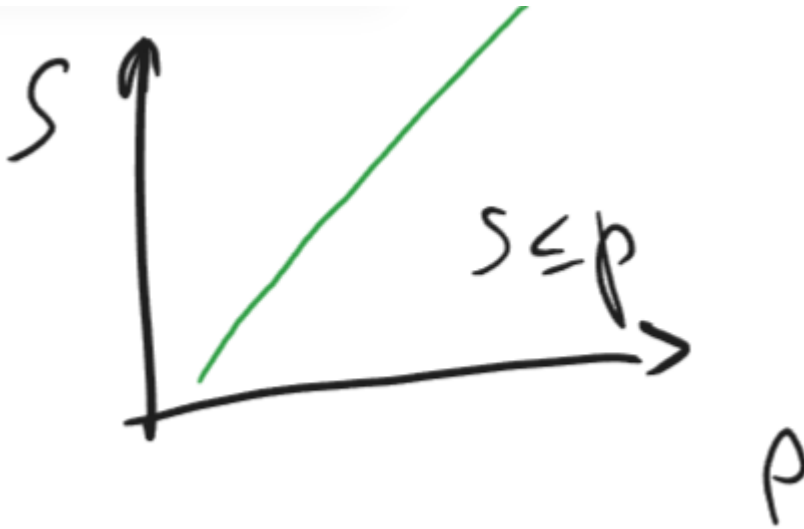
Рассмотрим крайности:

1. $p = \infty$

$$S \leq \frac{1}{\gamma}$$



2. $\gamma = 0$



30. Ускорение, эффективность, масштабируемость, учет коммуникаций при оценке эффективности, коэффициент сетевой деградации.

Ускорение (speedup): $S = \frac{T'}{T^p}$

T' - время работы последовательного

T^p - время работы параллельного, p - число задач

Смысл ускорения - во сколько раз вычисления по параллельному алгоритму производятся быстрее чем по последовательному

Эффективность (effecintly): $E = \frac{S}{p} = \frac{T'}{T^p p} * 100\%$

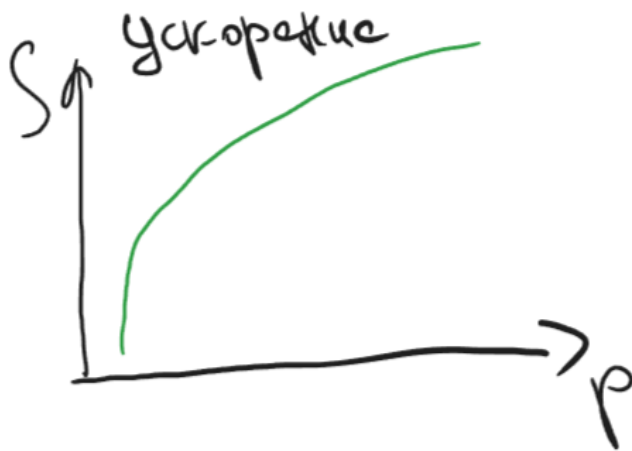
Смысл эффективности - указать какая доля исполнительных операций занимается полезными арифметическими операциями

Масштабируемость (scaleability): $M = \frac{V^p}{V'}$, при $T' = T^p$

V^p -объем памяти

V' - объем памяти при последовательном алгоритме

Смысл масштабируемости - это свойство алгоритма допускающее эффективное задействование большего количества исполнительных устройств с ростом объемов входных данных



Пункт 4.1.2.2 Учет коммуникаций

Пусть изучаемый параллельный алгоритм сбалансирован, то есть совокупное время арифметических операций и коммуникаций для любой из его задач одинакова, начало и конец вычислений производится всеми задачами одновременно и для всех задач одинакова длительность арифметических операций и коммуникаций.

При этом в любой произвольный момент времени разные задачи могут делать разное.

Если мы накладываем и это требование (в любой момент времени все делают одно и то же, пусть и над разными данными), то алгоритм называют **систолическим**.

Любой **систолический** алгоритм всегда **сбалансирован**, обратное неверно.

Далее исключительно о сбалансированных алгоритмах.

t_a - общее время производства арифметических операций для любой задачи

t_k - общее время производства коммуникаций для любой задачи

Тогда:

$$S = \frac{t_a * p}{t_a + t_k} = \frac{p}{1 + \frac{t_k}{t_a}}$$

k - коэффициент сетевой деградаций

$$E = \frac{1}{1+k}$$

Распишем коэффициент сетевой деградации:

$$k = \frac{t_k}{t_a} = \frac{q \hat{t}_k}{Q \hat{t}_a}$$

q - количество коммуникаций

$\hat{t}_k$ - время одной

Q - количество арифметических операций

$\hat{t}_a$ - время одной

Разделим на две части коэффициент:

$\frac{q}{Q}$ - алгоритмическая часть коэффициента сетевой деградации

$\frac{\hat{t}_k}{\hat{t}_a}$ - аппаратная часть коэффициента сетевой деградации

31. Правила постановки вычислительных экспериментов и анализа их результатов.

Пункт 4.1.3 Экспериментальные исследования параллельных алгоритмов

Если в ходе эксперимента производятся измерения физических величин, то он натурный;

Если расчет физических величин, то теоретический (вычислительный).

Ход исследования:

1. Поставить цель
2. Выбор параметров + обоснования
3. Выбор программной, аппаратной и системной базы
4. Предварительные теоретические предположения (либо вторым пунктом)
5. Постановка эксперимента (в итоге таблицы и графики) - **Необходим лабораторный журнал**
6. Описание результатов (словесное, а лучше письменное) - что происходит? как происходит?
7. Анализ (сравнение теории и эксперимента) - почему происходит?
 - 7а. Качественно и количественно соответствовать -> завершаем эксперимент
 - 7б. Качественно соответствовать, количественно не совпадать -> повод задуматься о выборе параметров, повторить или уточнить теорию
 - 7в. Никакого соответствия -> новая теория
- Феноменологический подход
8. Выводы (соответствие результата и цели)